

Adaptive DBSCAN-Enhanced Oversampling for Imbalanced Network Intrusion Detection: A Boundary-Aware Approach

Jingnan Dong¹, Member, IEEE, Boran Zhang², Student Member, IEEE, Shigen Shen³, Senior Member, IEEE, Jun Liu⁴, Member, IEEE, Guangxia Xu⁵, Senior Member, IEEE, and Zhiquan Liu⁶, Member, IEEE

Abstract—Class imbalance is one of the primary challenges faced by machine learning-based network intrusion detection systems (NIDS), particularly when identifying rare yet high-risk attack types. Traditional oversampling techniques, such as random oversampling and SMOTE, often disrupt the inherent manifold structure of the data and introduce noise near class boundaries, thereby degrading detection performance for minority classes. To address this problem, this paper introduces the ADEO framework—an adaptive DBSCAN-enhanced oversampling method that effectively mitigates these issues by combining deep reinforcement learning with boundary-aware geometric modeling. The framework utilizes a Deep Q-Network (DQN) to automatically optimize DBSCAN parameters, enabling accurate identification of natural clustering structures within heterogeneous intrusion datasets. Building upon this foundation, ADEO constructs precise class boundary functions using methods such as convex hulls, minimum enclosing ellipses, or axis-aligned bounding boxes. It then generates samples within these constrained safe regions, preserving the true topological structure while avoiding noise near decision boundaries. Furthermore, this paper introduces hierarchical class mapping strategies and attack-specific feature engineering to enhance the framework's adaptability to different attack families. Experimental results on the NSL-KDD, UNSW-NB15, and CICIDS2017 benchmark datasets demonstrate that ADEO outperforms mainstream oversampling methods in terms of precision, accuracy, recall, F1 score, FPR, and AUC. Particularly notable improvements are achieved for extreme minority classes like Backdoor and Heartbleed, validating

the proposed framework's effectiveness and efficiency in practical NIDS applications.

Index Terms—Network intrusion detection, oversampling, dataset imbalance, machine learning.

I. INTRODUCTION

WITH the rapid development of 5G communication technology and Big Data technology, an increasing number of IoT devices are being deployed at the edge to collect various types of information and data [1], ushering in an era of universal connectivity. The Internet of Things (IoT) utilizes advanced communication technologies to bridge the physical and digital worlds [2]. Businesses analyze the collected data to assist managers in decision-making. However, real-world networks are rife with various cyberattacks [3], [4]. To better defend against these attacks, it is necessary to monitor user access behavior, which has led to the emergence of network intrusion detection.

As a proactive defense technology against cyberattacks, network intrusion detection has been extensively studied [5]. In recent years, various network intrusion detection techniques have been proposed. The primary objective of network intrusion detection is to classify network access behavior into benign and malicious categories, and even to further subdivide different types of malicious behavior [6]. However, the frequency of many malicious behaviors is far lower than that of normal access behavior. Consequently, this leads to a class imbalance problem in network intrusion detection data [7]. In Network Intrusion Detection Systems (NIDS), the issue of class imbalance is particularly pronounced for complex attack types that occur infrequently but pose significant threats. Taking the UNSW-NB15 dataset as an example, critical attack types such as shellcode and worms account for less than 0.1% of the total samples; however, accurately identifying these minority samples is crucial for ensuring overall network security [8].

The primary methods for addressing data imbalance in network intrusion detection include oversampling techniques [9], undersampling techniques, hybrid sampling techniques, or synthetic sampling techniques. Oversampling techniques achieve class balance by replicating samples from the minority class to increase their number. Although this method is simple to implement and requires no assumptions about the data distribution, it merely replicates existing samples without introducing new

Received 3 January 2026; revised 6 May 2026; accepted 10 June 2026. Date of publication 15 June 2026; date of current version 1 September 2026. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB2703802, in part by the National Natural Science Foundation of China under Grant 62272120, in part by the Scientific Research Fund of Zhejiang Provincial Education Department under Grant Y202559768, in part by the Chongqing Research Program of Basic Research and Frontier Technology, China under Grant CSTB2025NSCQ-GPX1309, and in part by the Jointly Funded Projects of Universities, Institutes and Enterprises under the Basic Research Program of Guangzhou under Grant 2024A03J0325. (Corresponding authors: Boran Zhang; Guangxia Xu.)

Jingnan Dong, Boran Zhang, and Shigen Shen are with the School of Information Engineering, Huzhou Normal University, Huzhou 313000, China (e-mail: dongjingnan@zjhu.edu.cn; 2023162502@stu.zjhu.edu.cn; shigens@zjhu.edu.cn).

Jun Liu is with the School of Software Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: junliu@cqupt.edu.cn).

Guangxia Xu is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China, and also with the Department of New Networks, Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: xugx@gzhu.edu.cn).

Zhiquan Liu is with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: zqliu@jnu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3703615

discriminative information. Consequently, it can easily lead to a memory effect in the model regarding the repeated samples, thereby causing overfitting issues [10].

Random undersampling adjusts the class distribution by reducing the number of samples from the majority class, but this process inevitably results in the loss of potentially useful information within the majority class [11]. Furthermore, a significant reduction in the number of samples alters the original distribution structure of the data, causing shifts in the similarity relationships among samples. This may violate the classifier's original modeling assumptions, thereby affecting its generalization performance and prediction stability [12]. Hybrid sampling combines undersampling and oversampling techniques.

Synthetic sampling methods primarily generate minority class samples through geometric generation or by utilizing generative adversarial networks. A well-known geometric generation method is the Synthetic Minority Over-sampling Technique (SMOTE) [13]. In SMOTE, within the feature space, k nearest neighbors of the same class are randomly selected for each minority class sample, and a new synthetic sample is randomly generated on the line connecting that sample to one of its neighbors. This effectively expands the decision region for minority class samples, allowing the classifier to learn more generalized boundaries and thereby mitigating the issue of model prediction bias toward the majority class caused by class imbalance. The core of GAN-based minority class sample generation methods lies in the generative and discriminative components learning the distribution of real data through mutual interaction, thereby generating novel and diverse minority class samples to address the problem of class imbalance [14].

A. Motivation

Existing oversampling methods are prone to causing model overfitting because they simply replicate samples from minority classes. Meanwhile, SMOTE and its derivatives generate synthetic samples through linear interpolation in the feature space, which often fails to accurately capture the true nonlinear distribution of cyberattack data. This results in noisy samples near class boundaries, thereby blurring the gaps between classes. Minority class synthesis methods based on generative adversarial networks (GANs) suffer from training instability, higher computational costs, and poor performance in extremely imbalanced or low-sample-size scenarios, often resulting in model collapse or unreliable samples. To address these issues, this paper proposes a novel oversampling framework capable of accurately capturing class boundary distributions, preserving attack-specific features, and delivering high computational efficiency.

B. Contribution

Overall, this paper proposes a novel adaptive DBSCAN-enhanced oversampling framework (ADEO). The framework first utilizes a density-based spatial clustering method (DBSCAN) combined with a dynamic optimization strategy to accurately identify the clustering boundaries inherent to various attack patterns; subsequently, it employs geometric modeling techniques such as convex hulls and minimum enclosing ellipses

to construct precise sampling regions that align with the true data distribution. This boundary-aware mechanism ensures that synthetic samples are generated only within the safe range of the natural distribution of instances in a minority class, thereby fully preserving the discriminative features of different attack types while avoiding the introduction of noise.

The main contributions of this paper include the following:

- 1) Adaptive DBSCAN parameter optimization mechanism: This paper proposes a reinforcement learning strategy based on a Deep Q-Network (DQN) that can automatically determine the optimal neighborhood radius (ϵ) and minimum point threshold (minPts) in DBSCAN according to the characteristics of different datasets and attack categories.
- 2) Boundary-aware synthetic sample generation strategy: This paper designs a data sampling mechanism based on geometric boundary modeling, which precisely characterizes the geometric contours of clustered regions using techniques such as convex hulls, minimum enclosing ellipses, or axis-aligned bounding boxes. This strategy generates synthetic samples uniformly within the identified boundaries, effectively preserving the statistical characteristics of minority classes while avoiding the introduction of noise and adapting to cluster distributions of arbitrary shapes.
- 3) Feature enhancement methods tailored to specific attack categories: This paper constructs a feature engineering framework that integrates domain knowledge. Specifically targeting hard-to-detect minority attack classes (such as Fuzzers and Worms) in the UNSW-NB15 dataset, designing customized feature transformation and selection strategies. This mechanism effectively enhances the discriminative power of the feature space, thereby improving the model's ability to identify rare attack patterns.
- 4) Comprehensive and reproducible evaluation framework: The proposed method was evaluated against five representative classifiers and the latest relevant research methods on three public benchmark datasets. It demonstrated superior performance across multiple metrics, including class-specific accuracy, precision, recall, F1 score, False Positive Rate (FPR), and AUC, thereby validating its effectiveness and robustness.

The remaining sections are organized as follows. Section II provides a systematic review of relevant research. Section III details the proposed adaptive DBSCAN-enhanced oversampling framework. Section IV comprehensively introduces the experimental setup, comparison benchmarks, and results analysis. Section V summarizes the research results and outlines future research directions.

II. RELATED WORK

This section summarizes methods for handling imbalanced datasets in network intrusion detection, primarily including traditional oversampling methods and their improvements, clustering-enhanced oversampling methods, geometric and boundary-optimized oversampling methods, and oversampling

TABLE I
MULTI-DIMENSIONAL COMPARATIVE ANALYSIS OF RELEVANT OVERSAMPLING METHODS

Method	Generation Methodology (Geometric Vs. Generative)	Optimization Strategy (RL Vs. Heuristic)	Noise Handling	Reported Metrics
SMOTE [13]	Geometric method: Linear interpolation	Heuristic: Dynamically adjusting the sampling ratio	Not addressed	ROC curve, AUC
ADASYN [15]	Geometric Method: Adaptive Weighted Interpolation	Heuristic: Weighted Sampling Based on Difficulty Distribution	Not addressed	Accuracy, Precision, Recall, F1 score, G-mean
G-SMOTE [16]	Geometric Methods: Generating Hypersphere or Hyperellipsoid Spaces	Heuristics: domain selection strategies, pruning factors, transformation factors	Safe region constraints: Limit the geometric radius of the generated samples	AUC, F1 score, G-mean
PPFCM-SMOTE [17]	Geometric method: Linear interpolation	Iterative Optimization: Based on Probabilistic Fuzzy Clustering	Noise Identification Based on Membership and Typicality Matrices	Accuracy, Precision, ROC Curve, and AUC
GMF-SMOTE [18]	Generative: Gaussian Mixture Models (GMM)	Heuristic: Iterative optimization of GMM parameters using the EM algorithm	GMM-Based Probabilistic Noise Filtering	Sensitivity, Specificity, G-mean, MCC
CCO [19]	Geometric Methods: Generating Convex Hulls	Density-based clustering optimization	Cluster-based Kernel Filtering	Balanced Accuracy (BACC), Matthews' Coefficient of Agreement (MCC), F1 score, G-mean
NG-SMOTE [20]	Geometric Methods: Hypersurface Sampling and Interpolation	Heuristic: Integrated noise filtering, dynamic clustering, and golden section interpolation	Integrated Filtering: Based on SVM Classifiers and Error Rate Thresholds	Recall, AUC, F1 score, G-mean
EB-Smote [21]	Geometric Methods: Generating Convex Hulls	Geometric Methods: Generating Convex Hulls	Sample Categorization and Filtering	Precision, Accuracy, AUPRC, AUROC, Wilcoxon signed-rank test
TMG-GAN [8]	Generative: Generative Adversarial Networks	Heuristics: Multi-classifier architecture, cosine similarity loss function	Classifier filtering: Removing high-uncertainty noise samples	Precision, Recall, F1 score
MCGC-IDS [22]	Generative: Multi-Critic Generative Adversarial Network (Multi-Critics GAN)	Clustering Optimization: Weighted Fuzzy C-Means (WFCM)	Denoisy Autoencoder (DAE)	Accuracy, Precision, Recall, F1 score, ROC curve, AUC
GAN-SMOTE [23]	Generative: Generative Adversarial Networks (GANs)	Heuristics: Dynamic Parameter Adjustment and Network Structure Optimization	Not addressed	F1 score, AUC, MCC
SMOTE-CLS [24]	Geometric Methods: Interpolation	Heuristic: Sample partitioning based on classification difficulty	Multi-stage filtering: latent space density-based filtering, decision boundary optimization	AUPRC, AUC, F1 score, G-Mean
ADEO	Geometric Methods: Convex Hull, Minimum Enclosing Ellipse, and Axis-Aligned Bounding Box	Clustering Optimization: DBSCAN + RL	The noise penalty mechanism in the DQN reward function	Accuracy, Precision, Recall, F1 score, FPR, AUC, Wilcoxon signed-rank test

methods based on generative adversarial networks or variational autoencoders. In addition, we have conducted a comparative analysis of these methods in terms of generation approaches (geometric vs. generative), optimization strategies (RL vs. heuristic), noise handling, and reporting metrics, as detailed in Table I.

A. Traditional Oversampling Methods and Their Improvements

To address the issue of low recognition rates for minority classes in classification tasks involving imbalanced datasets, Chawla et al. [13] first proposed the SMOTE. However, SMOTE does not account for differences in the learning difficulty of minority class samples, resulting in insufficient attention to hard-to-learn samples. To address this, He et al. [15] proposed the Adaptive Synthetic Sampling method (ADASYN), whose core idea is to assign weights based on the learning difficulty of minority class samples, generating more synthetic data for harder-to-learn samples to adaptively adjust the classification decision boundary. Furthermore, to address the shortcomings

of SMOTE's linear interpolation mechanism—such as noise generation and lack of diversity—Douzas et al. [16] proposed the Geometric SMOTE (G-SMOTE) algorithm. This method generates synthetic samples by defining geometric regions (such as hyperspheres or hyperellipsoids) around each minority-class instance, replacing the traditional mechanism of generating samples along line segments.

B. Cluster-Enhanced Oversampling Methods

To reduce the interference of noisy samples in the oversampling process, researchers have combined clustering techniques with SMOTE. Manokaran et al. [17] proposed a hybrid balanced system (PPFCM-SMOTE) that combines Probabilistic Possibility Fuzzy C-Means (PPFCM) with SMOTE; this approach uses PPFCM to cluster data and reduce noise, and then employs SMOTE to generate synthetic samples. Xu et al. [18] proposed a Gaussian Mixture Model Filtering-based synthetic minority class oversampling technique (GMF-SMOTE), which groups data using the Expectation-Maximization algorithm to

filter out noisy and marginal samples, and designs a dynamic oversampling ratio for synthetic samples. To address the issue that SMOTE and its improved methods tend to generate noisy samples in non-convex complex data, Mondal et al. [19] proposed the Cluster Cores-based Oversampling technique (CCO), which identifies local high-density regions of minority class data and generates synthetic samples within the convex hull of each cluster.

C. Oversampling Methods for Geometric and Boundary Optimization

To address the issues of noise and outlier samples commonly associated with SMOTE-generated samples, researchers have proposed improvements from the perspectives of geometric space and boundary optimization. Sun et al. [20] introduced NG-SMOTE, an oversampling technique based on noise detection and geometric knowledge. This method eliminates noisy samples using an ensemble filter, clusters minority-class samples to obtain cluster centers, constructs a hyperplane between the cluster centers and the nearest majority-class samples, and generates new samples. During the same period, Sun et al. [21] also proposed the EB-SMOTE method, which selects boundary samples and generates extended synthetic samples. By oversampling both minority and majority classes simultaneously, it generates more informative samples in the boundary region to optimize the decision boundary.

D. Oversampling Methods Based on Generative Adversarial Networks or Variational Autoencoders

Generative Adversarial Networks (GANs) are widely used in imbalanced learning due to their powerful data generation capabilities. Ding et al. [8] proposed the TMG-GAN model, which constructs a multi-generator architecture to simultaneously generate different types of attack data, introduces a classifier to optimize the generators and discriminators, and uses cosine similarity as the generator loss to improve the quality of generated samples and reduce inter-class overlap. Wang et al. [22] proposed the Multi-Classifer Generative Adversarial Network-based Clustering Intrusion Detection System (MCGC-IDS) model. This approach uses a multi-classifier WGAN-GP to generate high-quality synthetic data to balance the dataset, combines it with a weighted fuzzy C-means clustering algorithm to group the data, and trains an independent CNN-LSTM model for each cluster.

Liu et al. [23] proposed a GAN-based improved SMOTE algorithm (GAN-SMOTE), which combines k nearest neighbor filtering of sample neighborhood points as the GAN's real-data input and uses adversarial training to generate minority-class samples that conform to the data distribution. Hong et al. [24] proposed an improved SMOTE framework (SMOTE-CLS) that integrates conditional latent-variable autoencoders (CLS). This approach quantifies data point density in a low-dimensional latent space and, after filtering out data points that might reduce the augmentation effect based on class labels and classification difficulty information, performs neighborhood sample augmentation directly in the data space.

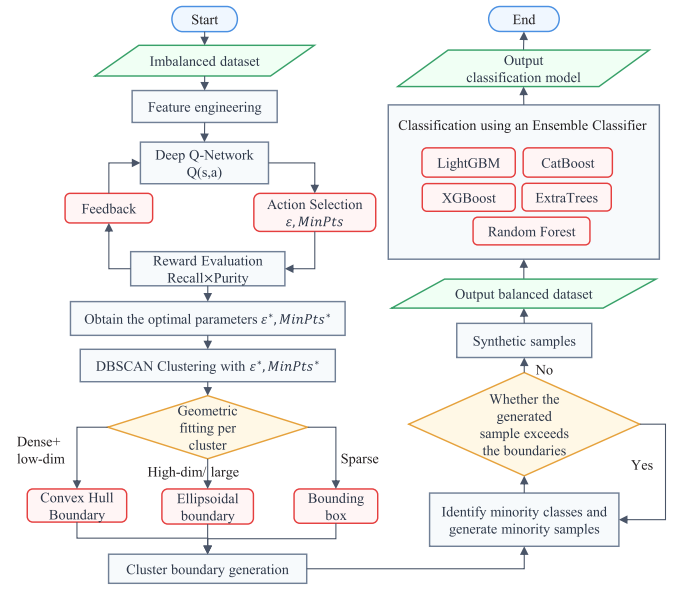


Fig. 1. Flowchart of the ADEO Framework.

Table I summarizes methods for synthesizing minority-class samples across four dimensions: generation methods, optimization strategies, noise handling, and evaluation metrics. Overall, existing methods primarily rely on geometric models (such as linear interpolation and hypersphere sampling), supplemented by a small number of generative adversarial networks. Optimization strategies are mainly heuristic, noise handling methods vary, and evaluation metrics focus on accuracy, precision, F1 score, recall, and G-Mean. The distinctive features and innovations of the proposed method in this paper are primarily reflected in the following aspects: flexible adoption of convex hulls, minimum enclosing ellipses, and axis-aligned bounding boxes to generate samples for datasets of varying scales; optimization of clustering results by combining DBSCAN with reinforcement learning; and noise filtering through a noise penalty mechanism in the DQN reward function. In terms of evaluation metrics, in addition to covering commonly used metrics, this method further incorporates AUC value of the ROC curve, FPR, and the Wilcoxon signed-rank test, significantly enhancing the comprehensiveness of the evaluation.

III. METHODOLOGY

A. System Architecture Overview

This study proposes the ADEO framework based on existing NIDS datasets, addressing the class imbalance problem in network intrusion detection through a combination of adaptive clustering and boundary-aware sampling. Unlike traditional re-sampling methods that operate globally within the feature space, ADEO identifies the natural data distribution of each attack class and generates synthetic samples only within the learned boundary range. This strategy effectively avoids introducing noise in critical decision regions while preserving the discriminative features of minority class samples.

TABLE II
HIERARCHICAL MAPPING RELATIONSHIPS IN INTRUSION DETECTION DATASETS

Dataset	Class IDs & Names
NSL-KDD	C0:Normal, C1:Dos, C2:Probe, C3:R2L, C4:U2R
UNSW-NB15	C0:Normal, C1:DoS/Generic, C2:Reconnaissance/Analysis, C3:Backdoor, C4:Shellcode/Exploits, C5:Fuzzers/Worms
CICIDS2017	C0:Normal, C1:DoS/DDoS, C2:PortScan, C3:Infiltration, C4:Bot, C5:Brute Force, C6:Web Attacks, C7:Heartbleed

The overall processing flowchart of the ADEO framework is illustrated in Fig. 1. Inputting an imbalanced dataset firstly, which is then processed by the ADEO framework to generate a balanced dataset. This balanced dataset is subsequently fed into an ensemble classifier comprising five classification methods—LightGBM, XGBoost, CatBoost, Random Forest, and ExtraTrees—which produces classification models via five-fold cross-validation. The ADEO framework consists of three tightly integrated components: (1) an adaptive DBSCAN parameter optimization module based on deep Q-learning; (2) a geometric modeling module focused on cluster boundaries; and (3) a uniform sampling and sample generation module under boundary constraints.

This system addresses three core challenges in network intrusion detection: heterogeneous attack distributions across datasets, severe class imbalance, and the difficulty of adaptive parameter selection in clustering-based data augmentation methods. The proposed approach effectively ensures the topological consistency between synthetic data and real attack patterns by constraining sample generation within learned cluster boundaries.

B. Adaptive DBSCAN Enhanced Oversampling Framework

Classifying attack types is crucial for building effective intrusion detection systems, as different attack families exhibit fundamental behavioral differences that require tailored countermeasures. For instance, reconnaissance attacks typically manifest as high port diversity and rapid connection attempts; backdoor attacks are characterized by persistent connections and asymmetric data flows; while exploitation attacks often exhibit irregular packet size distributions caused by payload injection. Because these behavioral signatures differ significantly, intrusion detection must perform attack-specific feature engineering to extract more discriminative key characteristics.

Furthermore, intrusion detection datasets commonly suffer from severe class imbalance, with some attack types accounting for less than 0.1% of samples. Therefore, a differentiated sampling strategy based on class-specific augmentation factors must be adopted.

To support precise threat response, fine-grained classification is also indispensable: for instance, port scanning requires network segmentation, while backdoor intrusions necessitate immediate endpoint isolation. To this end, this paper employs a hierarchical mapping relationship, as detailed in Table II, to aggregate attack categories across different datasets based on

behavioral consistency: UNSW-NB15 uses a 6-category taxonomy, merging Reconnaissance/Analysis and Fuzzers/Worms. Fuzzers and Worms both fall under destructive attacks in terms of motivation: the former triggers system anomalies by sending malformed packets, while the latter depletes network resources through self-replication and propagation. As both aim to disrupt normal system functionality, merging them into category C5 is semantically justified. NSL-KDD retains the standard five-category classification; CICIDS2017 retains its 8-category granular structure to preserve attack vector specificity. This classification directly guides subsequent DBSCAN clustering and boundary construction, ensuring synthesized samples accurately reflect attack-specific manifold geometry.

The specific execution process of the ADEO framework is as follows.

1) *Adaptive DBSCAN Parameter Optimization Based on Deep Q-Learning*: The first component employs reinforcement learning to automatically determine optimal DBSCAN parameters, adapting to the unique characteristics of different datasets. Traditional DBSCAN relies on manually setting the neighborhood radius (ϵ) and minimum point threshold (minPts), making its clustering quality highly sensitive to data density and dimensionality. This method models parameter selection as a Markov decision process, where the agent iteratively interacts with the environment to progressively learn optimal parameter configurations. The state representation encapsulates comprehensive dataset characteristics, including data distribution, local density structure, and dimensionality properties, the formal representation is shown in (1).

$$\mathbf{s}_t = [n, d, p_c, \mu_\sigma, d_k]^T \in \mathbf{R}^{D_s} \quad (1)$$

where n denotes the number of samples, d represents the feature dimension, p_c indicates the category distribution probability, μ_σ captures feature variability, and d_k refers to the k -nearest neighbor distance statistics including the mean $\bar{d}_k$ and standard deviation σ_{d_k} . The state dimension D_s varies with the number of classes C ($D_s = 2 + C + 8$).

The action space consists of ϵ candidate values sampled uniformly from a logarithmic scale, $a_t = (\epsilon, \text{minPts}_j)$, whose range is adaptively adjusted based on the density characteristics of the dataset (e.g., dense datasets may be set to $[10^{-2.5}, 10^{1.5}]$). This is combined with a set of integer minPts values (typically $\{3, 5, 7, 10, 15, 20, 25, 30\}$) to form a parameter combination, the formal representation is shown in (2).

The Deep Q-Network employs a three-layer fully connected architecture ($256 \rightarrow 128 \rightarrow 64$ neurons) to progressively abstract states and extract features layer by layer.

$$\mathbf{Q}(\mathbf{s}_t; \theta) = [\mathbf{Q}_\epsilon(\mathbf{s}_t; \theta), \mathbf{Q}_{\text{minPts}}(\mathbf{s}_t; \theta)] \quad (2)$$

$\mathbf{Q}(\mathbf{s}_t; \theta)$ represents the Q-function with parameters θ at state $\mathbf{s}_t$. $\mathbf{Q}_\epsilon(\mathbf{s}_t; \theta)$ denotes the Q-value output for the ϵ parameter. $\mathbf{Q}_{\text{minPts}}(\mathbf{s}_t; \theta)$: Q-value output with parameter minPts. θ represents neural network parameters.

The reward function R_t aims to jointly optimize multiple clustering quality metrics, with a focus on enhancing the identification performance of minority class samples, its formulaic

definition is (3).

$$R_t = w_r \cdot \text{Recall}_{avg} + w_p \cdot \text{Purity}_{avg} - w_n \cdot r_{noise} + w_c \cdot I_{C_{opt}} \quad (3)$$

w_r, w_p, w_n, w_c are weight coefficients, Purity_{avg} is the average purity, r_{noise} is the noise point ratio, $I_{C_{opt}}$ denotes the indicator function, where $I_{C_{opt}} \in \{0, 1\}$; $I_{C_{opt}}$ takes the value 1 if and only if $n_{classes} \leq n_{cluster_size} \leq n_{classes} * 3$, and 0 otherwise. Here, $n_{classes}$ denotes the number of categories, and $n_{cluster_size}$ denotes the cluster size.

The average recall rate is defined as (4).

$$\text{Recall}_{avg} = \frac{1}{C} \sum_{i=0}^{C-1} \frac{|\{x \in C_i : \text{cluster}(x) \neq -1\}|}{|C_i|} \quad (4)$$

C denotes the total number of categories, C_i represents the sample set of the i -th category, x denotes a single sample, and $\text{cluster}(x)$ indicates the cluster label assigned to sample x .

Average purity Purity_{avg} is defined (5).

$$\text{Purity}_{avg} = \frac{1}{C} \sum_{i=1}^C \frac{n_i}{|C_i|} \quad (5)$$

Here, $|C_i|$ denotes the total number of samples in that cluster, C denotes the number of classes, and n_i denotes the number of samples in that cluster belonging to class i .

Weight coefficients w_r, w_p, w_n, w_c employ a dataset-adaptive configuration mechanism: in scenarios where minority class detection is critical, recall-optimized weights are enhanced by increasing w_r . The training process combines experience replay with ε -greedy strategies. The resulting optimal parameters ($\varepsilon^*, \text{minPts}^*$) are applied to cluster analysis across the entire dataset. For large-scale datasets, a batch processing strategy is adopted to improve computational efficiency.

2) *Construction of Boundary Functions for Manifold Clustering*: After obtaining the optimal DBSCAN parameters, the second stage constructs boundary functions to characterize the geometric structure of each category-specific cluster. For each true category label y_i , the dominant cluster is determined by maximizing its intersection with the true sample distribution, its formal expression is given by (6).

$$C_i = \text{argmax}_{(c \in C)} |x_j : \text{cluster}(x_j) = c \wedge y_j = i| \quad (6)$$

$\text{argmax}_{(c \in C)}$: Among all possible cluster labels c , find the label c that maximizes the expression below, where y_j is the true category label of sample x_j .

Boundary function types are classified and adapted based on clustering characteristics (as shown in Fig. 2). For dense, low-dimensional clusters satisfying both the density condition ($n_c \geq n_{\text{threshold}}$) and the dimensionality condition ($d' \leq d_{\text{threshold}}$), convex hull boundaries are employed to precisely fit their data manifolds, the formal expression is shown in (7).

$$B_{\text{convex}}(C) = \left\{ x : x = \sum_{i=1}^{|V|} \lambda_i v_i, \sum_{i=1}^{|V|} \lambda_i = 1, \lambda_i \geq 0 \right\} \quad (7)$$

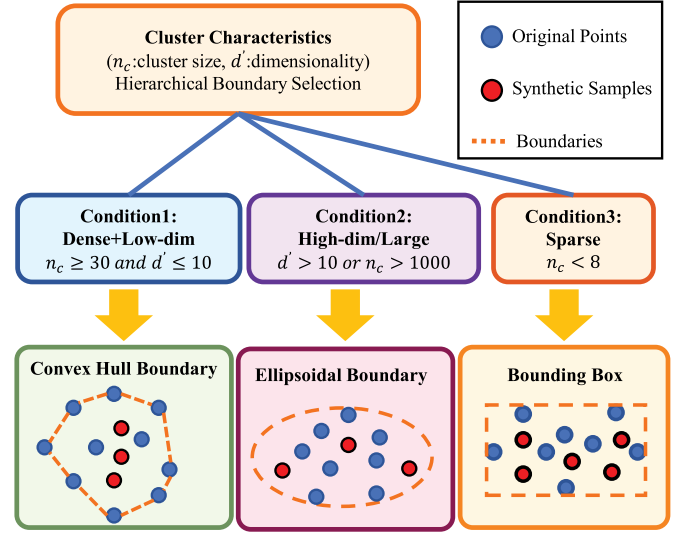


Fig. 2. Classification boundary selection strategy based on dataset examples.

$B_{\text{convex}}(C)$ denotes the convex hull boundary of the cluster, x represents any point within the convex hull, $v_i \in V(C)$ indicates the vertices of the convex hull, λ_i denotes the weight coefficient of the i -th vertex.

For high-dimensional or large-scale clustering, elliptical boundaries based on covariance structures are employed to enhance computational efficiency, as formally expressed in (8).

$$B_{\text{ellipse}}(C) = \left\{ x : (x - \mu_C)^T \Sigma_C^{-1} (x - \mu_C) \leq \tau^2 \right\} \quad (8)$$

Where $B_{\text{ellipse}}(C)$ denotes the elliptical boundary of the cluster, $\mu_C = \frac{1}{n_c} \sum_{x \in C} x$ represents the cluster center, where Σ_C denotes the covariance matrix, x represents any point within the convex hull, and τ is the threshold parameter for elliptical size. For attack categories with high variability, an expansion factor κ may be introduced to amplify the covariance matrix, thereby enhancing the coverage of the feature space.

For extremely sparse clusters ($n_c < n_{\text{min}}$), the axis-aligned bounding box is adopted as a robust alternative solution, formalized as shown in (9), $B_{\text{box}}(C)$ represents the bounding box of the cluster cluster.

$$B_{\text{box}}(C) = \prod_{j=1}^d \left[\min_{x \in C} x_j, \max_{x \in C} x_j \right] \quad (9)$$

3) *Generation of Synthetic Samples Under Boundary Constraints*: The third component synthesizes uniformly distributed samples within the constructed boundary to augment minority classes. For each minority class i where the current sample count n_i falls below the target value n_i^* , the framework generates $m_i = n_i^* - n_i$ synthetic samples strictly constrained within the boundary region B_i . Majority classes (satisfying $n_i \geq n_i^*$) retain their original distributions to ensure computational feasibility and maintain data balance.

The adaptive target distribution strategy adheres to three core principles:

- 1) *Extreme Minority Class Boosting*: For categories with an original proportion $\alpha_{\text{original}} < 0.1\%$, employ aggressive oversampling based on boundary constraints.
- 2) *Moderate Minority Class Balancing*: For classes satisfying $0.1\% \leq \alpha_{\text{original}} < 5\%$, apply proportional augmentation as shown in (10):

$$n_{\text{target}} = \min(\beta \cdot n_{\text{majority}}, \gamma \cdot n_{\text{original}}) \quad (10)$$

where $\beta \in [0.1, 0.5]$ maintains relative scarcity, $\gamma \in [2, 10]$ limits the synthetic amplification factor. n_{target} denotes the number of target samples in the target category, n_{majority} denotes the number of samples in the majority class, and n_{original} denotes the original number of samples in that category.

- 3) *Majority Class Preservation*: For original categories n_{majority} with a proportion $\alpha_{\text{original}} \geq 5\%$, their original distributions are maintained. The specific target quantities $\{n_i^*\}_{i=0}^{C-1}$ are determined by comprehensively considering the original category distribution, attack semantic similarity, and available computational resources.

For categories employing convex boundary shapes, uniform sampling is achieved by applying Dirichlet-weighted convex combinations to the vertex set, effectively leveraging the convex geometric structure, as shown in (11).

$$x_{\text{syn}} = \sum_{j=1}^k w_j v_{i_j}, w \sim \text{Dirichlet}(\alpha), k \in \{2, 3\} \quad (11)$$

x_{syn} denotes the generated synthetic sample, where k is the number of selected vertices. v_{i_j} is the j -th vertex randomly selected from the convex hull vertices. $w \sim \text{Dirichlet}(\alpha)$ indicates that the weight vector w follows a Dirichlet distribution, with α being the parameter vector of the Dirichlet distribution.

For categories with elliptical boundaries, synthetic samples are generated by sampling from a Gaussian distribution defined by the covariance matrix Σ_C , which has the cluster center μ_C as its mean and is scaled by the expansion factor κ , as shown in (12).

$$x_{\text{syn}} = \mu_C + \sqrt{\kappa} L z, z \sim \mathcal{N}(0, I), \Sigma_C = L L^T \quad (12)$$

L is the lower triangular matrix from the Cholesky decomposition such that $\Sigma_C = L L^T$, where Σ_C is the covariance matrix for cluster C .

By combining raw data with generated synthetic samples, a category-balanced dataset is constructed. This integration process preserves all original majority-class samples while supplementing each minority class with boundary-in synthetic data, ultimately forming an augmented dataset that maintains overall distribution equilibrium while preserving the original manifold structure, as shown in (13).

$$D_{\text{balanced}} = \text{Shuffle} \left(D_{\text{original}} \cup \bigcup_{i=1}^C S_i \right) \quad (13)$$

D_{balanced} denotes the balanced dataset, Shuffle refers to the randomization operation, D_{original} represents the original dataset, and S_i denotes the synthetic sample set for the i -th category.

- 4) *Attack-Specific Feature Engineering for Enhanced Discrimination*: To enhance detection performance and efficiency, this solution employs domain-knowledge-based feature engineering, customizing discriminative features for attack behaviors across three datasets. Beyond standard statistical aggregation, specialized features are designed to capture attack-specific patterns: For reconnaissance attacks (e.g., Probe in NSL-KDD, Reconnaissance/Analysis in UNSW-NB15, PortScan in CICIDS2017), port diversity metrics quantify scanning behavior, as shown in (14).

$$f_{\text{port}} = \sqrt{\frac{1}{n_{\text{ports}}} \sum_{i=1}^{n_{\text{ports}}} (p_i - \bar{p})^2} \quad (14)$$

Here, f_{port} denotes the port diversity index, n_{ports} represents the number of ports, p_i denotes the eigenvalue of the i -th port, and $\bar{p}$ denotes the mean.

For backdoor and R2L attacks, the asymmetry of data flow is quantified to measure the imbalance in bidirectional traffic, as specifically illustrated in (15).

$$f_{\text{asym}} = \frac{|B_{\text{src}} - B_{\text{dst}}|}{B_{\text{src}} + B_{\text{dst}} + \delta} \quad (15)$$

Where f_{asym} denotes the flow imbalance index, B_{src} represents the source (src) byte count, B_{dst} denotes the destination (dst) byte count, and δ represents a small real number ($1 * 10^{-10}$).

For the fuzzing class within the dataset, potential payload injection behavior is identified through irregularities in packet size distribution, as specifically illustrated in (16).

$$f_{\text{cv}} = \frac{\sigma_{\text{size}}}{\mu_{\text{size}} + \delta} \quad (16)$$

Here, f_{cv} represents the irregularity index of packet size distribution, σ_{size} denotes the standard deviation of packet size, μ_{size} indicates the mean of packet size, and δ represents a small real number ($1 * 10^{-10}$).

The complete feature vectors were processed using a robust normalization method based on the median and interquartile range, as detailed in (17).

$$\tilde{x}_j = \frac{x_j - \text{median}(X_j)}{IQR(X_j)} \quad (17)$$

$\tilde{x}_j$ denotes the standardized j -th feature value, x_j denotes the original j -th feature value, $\text{median}(X_j)$ denotes the median of X_j , $IQR(X_j)$ denotes the interquartile range of the feature X_j .

The top k features based on mutual information are selected for retention, where $k = \min(k_{\text{max}}, \alpha \cdot d_{\text{total}})$, with k_{max} being the preset maximum feature count (e.g., 250) and α representing the retention ratio factor (typically set between 0.75–0.8). Different datasets employ distinct parameter configurations based on their dimensional characteristics.

The feature space defined through the above methods significantly improves both DBSCAN clustering quality and ensemble classification performance across benchmark datasets, enabling the framework to effectively identify subtle attack

variations while maintaining high computational efficiency. Attack-specific features prove particularly crucial for distinguishing challenging categories, such as U2R/R2L in NSL-KDD, Fuzzers/Worms in UNSW-NB15, and Web attack sub-categories in CICIDS2017.

C. Complexity Analysis

The overall computational complexity of the ADEO framework primarily stems from the following five core components:

- 1) DBSCAN parameter optimization based on deep Q-learning.
- 2) Construction of clustering boundary functions.
- 3) Uniform sample generation under boundary constraints.
- 4) Discriminative feature engineering for attack categories.
- 5) Ensemble classifier training.

Therefore, the computational complexity of ADEO is as shown in (18).

$$T = T_1 + T_2 + T_3 + T_4 + T_5 \quad (18)$$

Let T_1, T_2, T_3, T_4 , and T_5 denote the computational costs of the five stages: parameter optimization, boundary construction, synthetic sampling, feature engineering, and ensemble training, respectively.

Among these, the complexity of T_1 can be expressed as shown in (19).

$$T_1 = O(T \cdot n_{sample} \cdot (k \cdot d + \log n_{sample})) \quad (19)$$

The computational complexity of T_2 is primarily influenced by the distribution of boundary types and the structural characteristics of each cluster, as detailed in (20).

$$T_2 = O(C \cdot n_c \cdot d'^2) \quad (20)$$

Let C denote the number of categories, n_c denote the average cluster size, and d' denote the feature dimension after dimensionality reduction. The computational complexity of elliptical boundaries is $O(n_c \cdot d'^2)$, dominating the overall cost; in contrast, the contributions from convex hull boundaries $O(n_c \cdot \log n_c)$ and bounding box functions $O(n_c \cdot d')$ are relatively minor.

During the boundary-aware sampling phase, the computational complexity (T_3) primarily consists of two components: boundary function construction and synthetic sample generation, as shown in (21).

$$T_3 = O(C \cdot (n_c \cdot d'^2 + d'^3) + M \cdot d'^2) \quad (21)$$

Among these, $O(C \cdot n_c \cdot d'^2)$ denotes the covariance computation cost required to construct the boundary functions for C categories. $O(C \cdot d'^3)$ represents the overall cost of matrix decomposition for elliptical boundaries across C classes, where each elliptical boundary requires either covariance matrix inversion or Cholesky decomposition. $O(M \cdot d'^2)$ denotes the sampling cost for generating M synthetic samples of dimension d . Multivariate normal distribution sampling requires matrix-vector multiplication, with a per-sample generation complexity of $O(d'^2)$.

The computational complexity of stage T_4 is primarily dominated by the feature selection process based on mutual information, as specifically demonstrated in (22).

$$T_4 = O(n \cdot d_{total} \cdot \log d_{total}) \quad (22)$$

Here, n denotes the sample size, and $d_{total} = d + d_{stat} + d_{attack}$ represents the total feature dimension after integrating statistical features and attack-specific features onto the original features.

The computational complexity of stage T_5 is primarily determined by the ensemble training process, whose expression is shown in (23):

$$T_5 = O(N_{models} \cdot n_{trees} \cdot n_{train} \cdot d_{selected} \cdot \log n_{train}) \quad (23)$$

Here, $N_{models} = 5$ denotes the number of base learners in the ensemble, $n_{train} \approx 200,000$ represents the maximum training sample size constrained by memory limitations, $d_{selected} \approx \{n \in \mathbf{Z} | 200 \leq n \leq 250\}$ denotes the dimension after feature selection, while $n_{trees} = \{n \in \mathbf{Z} | 50 \leq n \leq 100\}$ represents the number of trees contained in each base model.

In summary, the overall time complexity of the model is as shown in (24).

$$\begin{aligned} T &= T_1 + T_2 + T_3 + T_4 + T_5 \\ &= O(T \cdot n_{sample} \cdot (k \cdot d + \log n_{sample})) + O(C \cdot n_c \cdot d'^2) \\ &\quad + O(C \cdot (n_c \cdot d'^2 + d'^3) + M \cdot d'^2) \\ &\quad + O(n \cdot d_{total} \cdot \log d_{total}) \\ &\quad + O(N_{models} \cdot n_{trees} \cdot n_{train} \cdot d_{selected} \cdot \log n_{train}) \end{aligned} \quad (24)$$

Among these, n_{sample} , $C \cdot n_c \cdot d'^2$, and $M \cdot d'$ are all significantly smaller than $n \cdot d_{total} \cdot \log d_{total}$ and $n_{train} \cdot d_{selected} \cdot \log n_{train} \cdot n_{trees}$, so the final time complexity of ADEO is determined by the following in (25).

$$T = O(n \cdot d_{total} \cdot \log d_{total} + N_{models} \cdot n_{trees} \cdot n_{train} \cdot d_{selected} \cdot \log n_{train}) \quad (25)$$

IV. EXPERIMENTAL COMPARATIVE ANALYSIS

A. Software and Hardware Experimental Environment

This experiment was conducted on a Windows 11 platform equipped with an Intel i9-13980HX (2.20 GHz) processor, featuring 32 GB of RAM, a 1 TB SSD, and an 8 GB NVIDIA GeForce RTX 4070 laptop GPU. The experimental framework was implemented in Python 3.11 and relied on the following key software libraries: PyTorch 2.8.0 was used to build the DQN network for optimizing DBSCAN parameters; Scikit-learn 1.6.1 was employed to implement clustering algorithms and feature processing workflows; LightGBM 4.6.0, XGBoost 3.0.0, and CatBoost 1.2.8 were integrated as base classifiers; imbalanced-learn 0.13.0 was used to reproduce the oversampling algorithm in the comparison method; Pandas 2.2.3 and NumPy 2.2.5 were employed for data loading, preprocessing, and numerical computations, respectively.

B. Datasets and Evaluation Measures

This section systematically compares the performance of the proposed ADEO method with other oversampling techniques on network intrusion detection datasets. The proposed approach applied 5-fold cross-validation to each experiment on three public datasets: NSL-KDD [25], UNSW-NB15 [26], and CICIDS2017 [27]. To comprehensively evaluate the effectiveness of different approaches, primarily employing the following five evaluation metrics: Accuracy (26), Precision (27), F1 score (28), and Recall (29), FPR (30), and AUC (31).

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (26)$$

$$Precision = \frac{TP}{TP + FP} \quad (27)$$

$$F1score = \frac{2 \times TP}{2 \times TP + FN + FP} \quad (28)$$

$$Recall = \frac{TP}{TP + FN} \quad (29)$$

$$FPR = \frac{FP}{FP + TN} \quad (30)$$

$$AUC = \int_0^1 TPR(FPR)d(FPR), TPR = \frac{TP}{TP + FN} \quad (31)$$

Specifically, Accuracy represents the proportion of correctly classified samples out of the total samples, reflecting overall classification performance. F1 score is the harmonic mean of precision and recall, providing a comprehensive measure of a model's performance when handling imbalanced data. Recall measures the model's ability to identify true positives, which is particularly crucial in scenarios sensitive to missed detections. FPR represents the proportion of normal traffic incorrectly classified as an attack, reflecting the model's false positive rate. In NIDS scenarios, FPR is a critical performance metric. An excessively high FPR leads to a large number of false alarms, which impacts the system's practical usability and operational costs. AUC, on the other hand, calculates the area under the ROC curve to comprehensively characterize the model's ability to distinguish between positive and negative samples at different decision thresholds. It provides an overall assessment of the model's discriminative performance and exhibits strong robustness against class imbalance. Together, these metrics form a comprehensive evaluation framework from multiple perspectives, objectively reflecting the overall performance of various models in intrusion detection tasks.

During the experiment, to fully utilize 32 GB of memory and process large-scale datasets comprehensively, the hierarchical mapping strategy (as Table II) is adopted. Specifically, the data loading module was configured with a block read size of 10,000 rows: the NSL-KDD dataset (containing 148,516 samples) could be loaded entirely into memory directly; while the larger UNSW-NB15 (2,540,047 samples) and CICIDS2017 (2,830,743 samples) datasets were processed in full through a chunking mechanism. This strategy effectively prevented memory overflow

while ensuring data integrity, providing the necessary foundation for the smooth execution of the ADEO framework.

The proposed approach was also compared with methods described in other papers, such as the data augmentation method TMG-GAN [8] based on generative adversarial networks, and the generative oversampling method called GOIO (Generative Oversampling Approach to Class Imbalance and Overlap) proposed by Ren et al. [28], which aims to simultaneously address class imbalance and class overlap in tabular data classification. GOIO combines a metric learning-based variational autoencoder (MLVAE) with a conditional latent diffusion model (CLDM).

C. Experimental Configuration

The state vector dimension D_s of the DQN-DBSCAN optimizer is $D_s = 2 + C + 8$, consisting of six statistical measures (mean, standard deviation, 90th percentile, 95th percentile, median, and maximum) of the sample count, feature dimension, proportion of each class, and k-NN distance. The action space is composed of a log-uniform candidate interval of ε and a discrete set of minPts. Training employs ε -greedy search and experience replay, with reward weights adapted to the dataset. Each dataset is trained for a fixed number of 15 episodes, and the final parameters are selected as $(\varepsilon^*, minPts^*)$ with the highest cumulative reward in history.

The boundary function is selected hierarchically based on the cluster size n_c and the dimension of the reduced space d' : when the density and dimensionality conditions are satisfied, the convex hull $B_{convex}(C)$ from (7) is used, and sampling is performed via Dirichlet-weighted vertex combination; for medium-sized clusters, this degenerates into the ellipse $B_{ellipse}(C)$ from (8). The expansion coefficient κ is configured by class (in UNSW-NB15, $\kappa = 2.5$ for class C5 and $\kappa = 2.0$ for standard classes), and for extremely sparse classes, it further degenerates into the axis-aligned bounding box $B_{box}(C)$ from (9).

The target distribution is divided into three segments based on the original proportion $\alpha_{original}$:

- 1) For the extreme minority class ($\alpha < 0.1\%$), aggressive oversampling is performed and diversity noise is injected.
- 2) Moderately minor classes ($0.1\% \leq \alpha < 5\%$) are scaled up according to the formula in (10), where $n_{target} = \min(\beta \cdot n_{majority}, \gamma \cdot n_{original})$ ($\beta \in [0.1, 0.5]$, $\gamma \in [2, 10]$)
- 3) Major classes ($\alpha \geq 5\%$) remain unchanged.

For the C5 class in UNSW-NB15, five additional targeted enhancements (such as packet size variation and frequency perturbation) are applied.

Feature engineering extracts two types of features: basic statistics (mean, standard deviation, quantiles, IQR, and approximations of skewness and kurtosis), as well as attack-specific features—port diversity for reconnaissance attacks f_{port} (14), data asymmetry for backdoor attacks f_{asym} (15), and packet size coefficient of variation for exploitation attacks f_{cv} (16). For the C5 class in UNSW-NB15, additionally calculating the fuzzing coefficient of variation, packet size distribution, and propagation intensity. All features are standardized using the RobustScaler in (17), and the top-k dimensions are selected based on mutual information.

TABLE III
ADEO OVERALL PERFORMANCE METRICS

Dataset	Accuracy	Precision	Recall	F1 score	FPR	AUC
NSL-KDD	99.41%±0.0003	99.41%±0.0003	99.41%±0.0003	0.9940±0.0003	0.0022±0.0001	0.9992±0.0001
UNSW-NB15	99.06%±0.0011	99.04%±0.0012	99.06%±0.0011	0.9904±0.0012	0.0059±0.0007	0.9988±0.0003
CICIDS2017	99.82%±0.0003	99.81%±0.0003	99.82%±0.0003	0.9981±0.0003	0.0008±0.0001	0.9992±0.0008

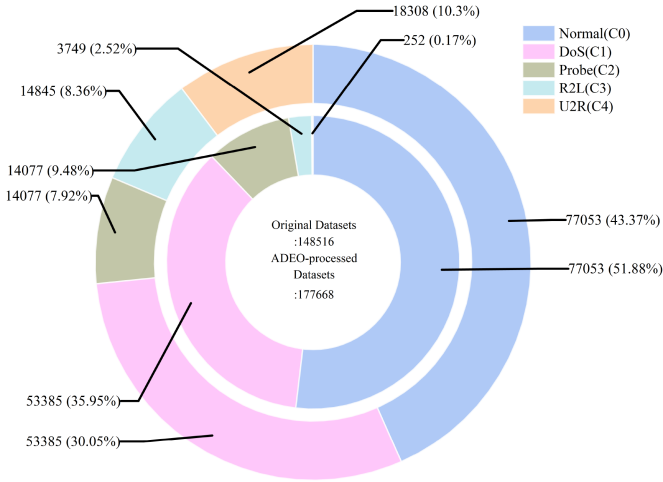


Fig. 3. Comparison of Distribution Characteristics Before and After Data Processing for the NSL-KDD Dataset.

The ensemble classifier combines five base learners, with weights $w_i = (accuracy_i + F1_i)/2$, and ultimately makes predictions via weighted soft voting. Batch processing is used for large-scale datasets to avoid memory overflow.

D. Experimental Results and Performance Evaluation

1) *Overall Performance*: Table III demonstrates the overall performance of ADEO across three network intrusion detection datasets. Experimental results indicate that the ADEO framework exhibits outstanding classification performance when handling datasets of varying scales and degrees of imbalance.

On the NSL-KDD dataset, ADEO achieved a precision of 99.41%, with recall and F1 score showing high consistency (99.41%/99.41%/0.9940) and a standard deviation of only ± 0.0003 , indicating that the model exhibits exceptional stability in five-fold cross-validation. With a FPR of 0.0022 and an AUC of 0.9992, this framework not only demonstrates excellent classification accuracy on this classic benchmark but also exhibits an extremely low false positive rate, showcasing outstanding generalization capabilities. Fig. 3 illustrates the original data distribution (inner ring) and the distribution after processing by the proposed method (outer ring). Through intelligent parameter optimization and boundary-constrained sampling strategies, ADEO effectively mitigates classification difficulties caused by sample scarcity, significantly enhancing detection capabilities for minority classes.

On the UNSW-NB15 dataset, the overall accuracy was 99.06%, which was the lowest among the three datasets. This was primarily due to the fact that the dataset contained six attack categories, several of which (such as Fuzzers/Worms

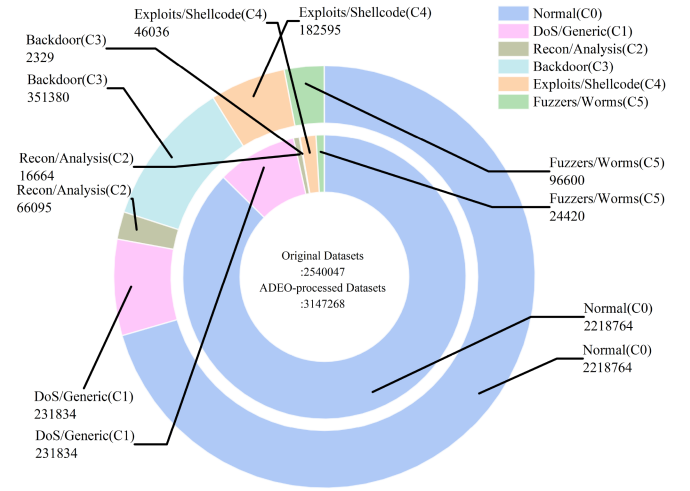


Fig. 4. Comparison of Distribution Before and After Data Processing for the UNSW-NB15 Dataset.

and Exploits/Shellcode) overlapped significantly in the feature space, increasing the difficulty of classification. Furthermore, its standard deviation ranges from 0.0003 to 0.0012, reflecting more pronounced performance fluctuations across different folds. Nevertheless, with an FPR of 0.0059 and an AUC as high as 0.9988, the model still demonstrates excellent discriminative capability in terms of probability ranking. Fig. 4 illustrates the distribution of the original data (inner ring) and the data processed by the proposed method (outer ring). The results demonstrate that ADEO maintains outstanding robustness and generalization capabilities when confronting complex attack patterns and extreme class imbalance.

On the CICIDS2017 dataset, ADEO performed exceptionally well, achieving an accuracy of 99.82%, an F1 score of 0.9981, a FPR as low as 0.0008, and an AUC of 0.9992. Although this dataset contains eight categories and has the largest total number of samples, its primary attack types (such as DoS/DDoS, PortScan, and Brute-Force) exhibit distinct traffic characteristics. ADEO's feature engineering module effectively extracts discriminative information, thereby achieving near-perfect overall classification performance. Fig. 5 illustrates the distribution of the raw data (inner ring) and the distribution of the data processed by the proposed method (outer ring). Even under such extreme oversampling conditions, ADEO maintains excellent classification performance, fully validating the effectiveness of the boundary-constrained sampling strategy while ensuring the geometric rationality and discriminative power of the synthesized samples.

2) *Performance Analysis by Category*: To thoroughly evaluate the effectiveness of ADEO in minority class detection, the

TABLE IV
ADEO'S PERFORMANCE METRICS ACROSS CATEGORIES ON THE NSL-KDD DATASET

Category	Number	Accuracy	Precision	Recall	F1 score	FPR	AUC	Percentage
Normal(C0)	77,053	99.72%±0.0001	99.18%±0.0005	99.72%±0.0001	0.9945±0.0003	0.0089±0.0006	0.9998±0.0000	51.88%
DoS(C1)	53,385	99.94%±0.0002	99.88%±0.0004	99.94%±0.0002	0.9991±0.0002	0.0007±0.0002	1.0000±0.0000	35.95%
Probe(C2)	14,077	99.05%±0.0013	98.96%±0.0007	99.05%±0.0013	0.9900±0.0004	0.0011±0.0001	0.9999±0.0000	9.48%
R2L(C3)	3,749	88.72%±0.0095	99.58%±0.0015	88.72%±0.0095	0.9383±0.0056	0.0001±0.0000	0.9991±0.0002	2.52%
U2R(C4)	252	69.42%±0.0822	93.93%±0.0475	69.42%±0.0822	0.7965±0.0680	0.0001±0.0001	0.9972±0.0006	0.17%

TABLE V
ADEO'S PERFORMANCE METRICS ACROSS CATEGORIES ON THE UNSW-NB15 DATASET

Category	Number	Accuracy	Precision	Recall	F1 score	FPR	AUC	Percentage
Normal (C0)	2,218,764	99.65%±0.0006	99.57%±0.0006	99.65%±0.0006	0.9961±0.0004	0.0300±0.0039	0.9997±0.0001	87.35%
DoS/Generic (C1)	231,834	99.37%±0.0009	99.96%±0.0001	99.37%±0.0009	0.9967±0.0004	0.0000±0.0000	1.0000±0.0000	9.13%
Recon/Analysis (C2)	16,664	94.14%±0.0070	91.65%±0.0344	94.14%±0.0070	0.9283±0.0156	0.0006±0.0003	0.9998±0.0001	0.66%
Backdoor (C3)	2,329	91.37%±0.0154	96.14%±0.0682	91.37%±0.0154	0.9356±0.0348	0.0000±0.0001	0.9987±0.0005	0.09%
Exploits/Shellcode (C4)	46,036	93.46%±0.0101	90.06%±0.0301	93.46%±0.0101	0.9171±0.0191	0.0019±0.0006	0.9994±0.0002	1.81%
Fuzzers/Worms (C5)	24,420	57.41%±0.0868	64.28%±0.0466	57.41%±0.0868	0.6042±0.0618	0.0031±0.0005	0.9954±0.0014	0.96%

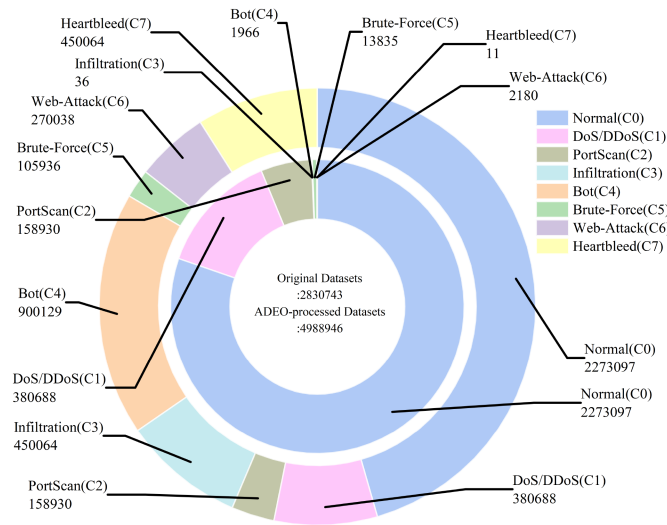


Fig. 5. Comparison of Distribution Characteristics Before and After Data Processing in the CICIDS-2017 Dataset.

proposed method conducts a detailed analysis of performance metrics across all categories for each dataset. Tables IV, V, and VI respectively present the performance metrics of the ADEO framework across all categories on three datasets, including accuracy, precision, recall, F1 score, FPR, and AUC.

On the NSL-KDD dataset, Normal (C0) and DoS (C1)—the two categories with the largest sample sizes (accounting for 51.88% and 35.95%, respectively)—both achieved near-perfect classification performance. The DoS category achieved a accuracy of 99.94% and an F1 score of 0.9991, with an FPR of only 0.0007, indicating that the model possesses extremely high accuracy in identifying large-scale denial-of-service attacks. Although the Probe (C2) category's sample proportion dropped to

9.48%, it still maintained an accuracy of 99.05% and an F1 score of 0.9900, demonstrating that ADEO's feature engineering module effectively captured the behavioral characteristics of probe attacks. For the R2L (C3) category, recall dropped to 88.72%, but precision reached 99.58% with an F1 score of 0.9383. Given that this category accounts for only 2.52% of the samples, these results indicate that ADEO's DBSCAN-enhanced oversampling strategy plays a significant role in low-resource categories. U2R (C4) is the most challenging class, with only 252 samples (0.17%). The recall rate is 69.42%, and the standard deviation reaches 0.0822, indicating significant fluctuation; however, the precision remains at 93.93%, with an AUC of 0.9972. This indicates that while the model may miss some instances in this extremely rare class, the reliability of samples predicted as U2R is very high. These results reveal a positive correlation between the degree of class imbalance and classification difficulty, while also validating the effectiveness of the ADEO framework in extremely imbalanced scenarios.

In the UNSW-NB15 dataset, the Normal (C0) class accounts for 87.35% of the samples, with an accuracy of 99.65%, however, its FPR is 0.0300, which is the highest among all Normal classes across the three datasets. This reflects a greater overlap in features between attack traffic and normal traffic in UNSW-NB15. DoS/Generic (C1) achieved a precision of 99.37% and an F1 score of 0.9967, with an FPR close to zero, indicating that the model has a very clear classification boundary for this attack category. Recon/Analysis (C2) and Backdoor (C3), as moderately rare categories (accounting for 0.66% and 0.09% respectively), achieved accuracy rates of 94.14% and 91.37%, respectively, with F1 scores around 0.93, demonstrating good classification performance. Notably, the Backdoor category achieved a precision rate of 96.14% despite having only 2,329 samples, highlighting the effectiveness of ADEO's synthetic sample generation strategy. Exploits/Shellcode (C4), accounting for 1.81%,

TABLE VI
ADEO'S PERFORMANCE METRICS ACROSS CATEGORIES ON THE CICIDS2017 DATASET

Category	Number	Accuracy	Precision	Recall	F1 score	FPR	AUC	Percentage
Normal (C0)	2,273,097	99.91%±0.0002	99.87%±0.0002	99.91%±0.0002	0.9989±0.0002	0.0052±0.0009	0.9998±0.0000	80.90%
DoS/DDoS (C1)	380,688	99.61%±0.0030	99.79%±0.0004	99.61%±0.0030	0.9970±0.0007	0.0003±0.0001	1.0000±0.0000	13.45%
PortScan (C2)	158,930	99.83%±0.0006	99.39%±0.0004	99.83%±0.0006	0.9960±0.0004	0.0004±0.0000	0.9999±0.0000	5.61%
Infiltration (C3)	36	72.14%±0.0915	97.14%±0.0571	72.14%±0.0915	0.8216±0.0484	0.0000±0.0000	0.9948±0.0061	0.00%
Bot (C4)	1,966	51.94%±0.0983	71.45%±0.1423	51.94%±0.0983	0.5821±0.0436	0.0002±0.0001	0.9991±0.0004	0.07%
Brute-Force (C5)	13,835	99.60%±0.0015	99.91%±0.0008	99.60%±0.0015	0.9975±0.0008	0.0000±0.0000	1.0000±0.0001	0.49%
Web-Attack (C6)	2,180	87.29%±0.0101	98.91%±0.0044	87.29%±0.0101	0.9274±0.0057	0.0000±0.0000	0.9997±0.0002	0.08%
Heartbleed (C7)	11	100%±0.0000	100%±0.0000	100%±0.0000	1.0000±0.0000	0.0000±0.0000	1.0000±0.0000	0.00%

achieved a precision rate of 93.46%, but its recall rate (90.06%) was relatively low, suggesting some feature overlap between this class and other attack categories. The most challenging category is Fuzzers/Worms (C5), which accounts for only 0.96% of the total, with an accuracy rate dropping to 57.41%, an F1 score of 0.6042, and a standard deviation as high as 0.0618. This result indicates that the traffic characteristics of fuzzing and worm attacks significantly overlap with those of other categories (particularly Exploits/Shellcode), highlighting an area where the current framework requires further improvement. Nevertheless, the AUC for C5 still reached 0.9954, indicating that the model can still effectively distinguish this category at the probabilistic level.

Table VI presents ADEO's detailed performance across the eight categories of the CICIDS2017 dataset. The "Normal" (C0) and "DoS/DDoS" (C1) categories, which account for 80.90% and 13.45% of the dataset respectively, both achieved accuracy rates exceeding 99.6% and F1 scores above 0.9970, indicating that the framework possesses highly reliable classification capabilities for mainstream traffic types. PortScan (C2), accounting for 5.61% of the data, achieved a 99.83% accuracy rate and an F1 score of 0.9960, with a precision of 99.39%. This is attributed to the distinct traffic pattern characteristics of port-scanning attacks. Although Brute-Force (C5) and Web-Attack (C6) had relatively few samples (accounting for 0.49% and 0.08%, respectively), the former achieved a 99.60% accuracy rate, while the latter reached an 87.29% recall rate and a 98.91% precision rate, indicating that ADEO's oversampling mechanism effectively enhances the recognition capabilities for these minority categories. Heartbleed (C7) had only 11 samples but achieved 100% perfect classification. While this is partly attributed to the highly distinctive signature of this attack type, it also demonstrates that ADEO can maintain effective learning even under extreme sample scarcity. However, Infiltration (C3) and Bot (C4) represent two significant weak points. Infiltration has only 36 samples, with a recall rate of 72.14% and a fluctuation as high as ± 0.0915 ; Bot had 1,966 samples but achieved a recall rate of only 51.94% and an F1 score of 0.5821. The poor performance of the Bot category is particularly concerning, as its sample size is not extremely scarce. The root cause lies in the fact that botnet traffic is highly similar to normal traffic in the feature space, characterized by low rates and long durations, making it difficult to distinguish from legitimate traffic. Nevertheless, the AUC for these two categories are 0.9948 and 0.9991, respectively,

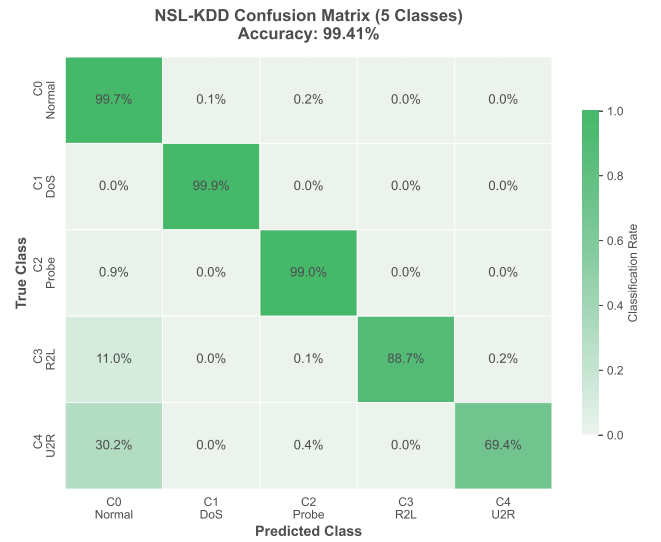


Fig. 6. Confusion Matrix of ADEO on the NSL-KDD Dataset.

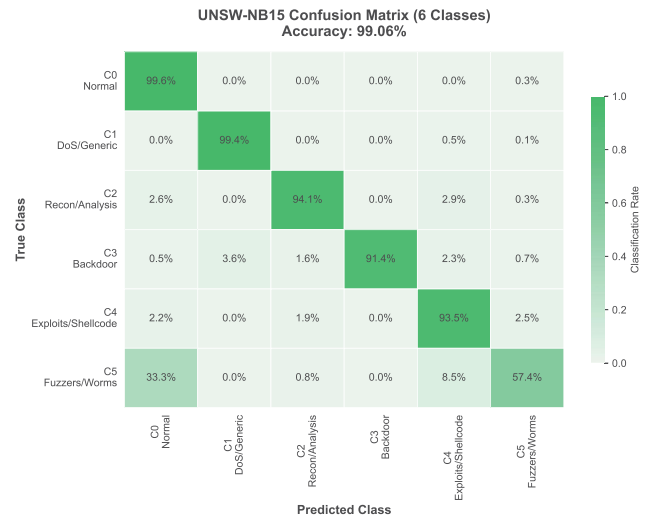


Fig. 7. Confusion Matrix of ADEO on the UNSW-NB15 Dataset.

indicating that the model still performs well in probabilistic ranking. In the future, recall can be improved by adjusting the decision threshold or introducing more granular features.

3) *Confusion Matrix and Visualization*: Figs. 6, 7, and 8 show the normalized confusion matrices for ADEO across

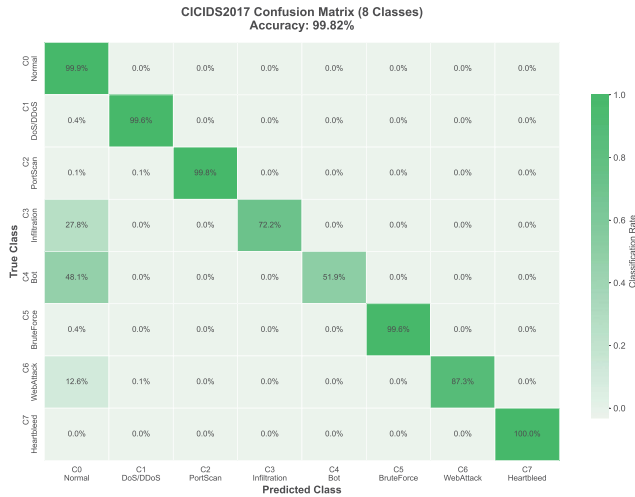


Fig. 8. Confusion Matrix of ADEO on the CICIDS2017 Dataset.

three datasets, respectively. The diagonal elements represent the correct classification rates (recall) for each category, while the off-diagonal elements represent misclassifications.

On the NSL-KDD dataset, the accuracy rates for the Normal, DoS, and Probe classes reached 99.7%, 99.9%, and 99.0%, respectively, while those for the minority classes R2L and U2R were only 88.7% and 69.4%, with 30.2% of U2R samples misclassified as Normal. On the UNSW-NB15 dataset, the accuracy for the Backdoor class was 93.5%, and for Exploits/Shellcode it was only 57.4%, while the Normal class achieved 99.6% with an extremely low misclassification rate. This indicates that the model is relatively robust against common attack types but remains susceptible to confusion when faced with certain complex attacks. On the CICIDS2017 dataset, the accuracy for the vast majority of classes exceeded 99%, with the outliers Infiltration and Heartbleed achieving 72.2% and 100%, respectively. Misclassifications primarily occurred between attack types with similar semantics (such as Bot).

Figs. 9, 10, and 11 show the distribution of original samples (solid dots) and synthetic samples (crosses) in the global two-dimensional t-SNE embedding space on the three benchmark datasets: NSL-KDD, UNSW-NB15, and CICIDS2017. From a structural perspective, all three figures demonstrate clear inter-class separability: each category clusters into compact and mutually isolated clusters, while the synthetic samples generated by ADEO are almost completely embedded within the original distribution of their corresponding categories. They neither form stray outliers outside the clusters nor exhibit cross-class drift, indicating that the proposed boundary-constrained sampling mechanism strictly preserves the manifold topology of the original data during the oversampling process. Specifically, in the Fig. 9, the four categories—Normal, DoS, Probe, and R2L—form distinct, large clusters, while the extremely rare category U2R (which, after boundary-internal sampling, coalesces into a small, compact cluster on the right) shows high overlap between synthetic and original samples, verifying that ADEO can stably generate geometrically reasonable samples even under conditions of extreme sample scarcity. In the Fig. 10, the

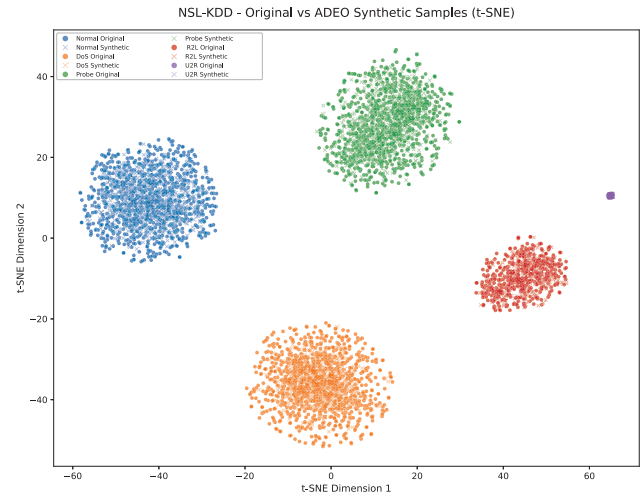


Fig. 9. Schematic of the distribution of synthetic samples generated by ADEO on the NSL-KDD dataset compared to the original samples.

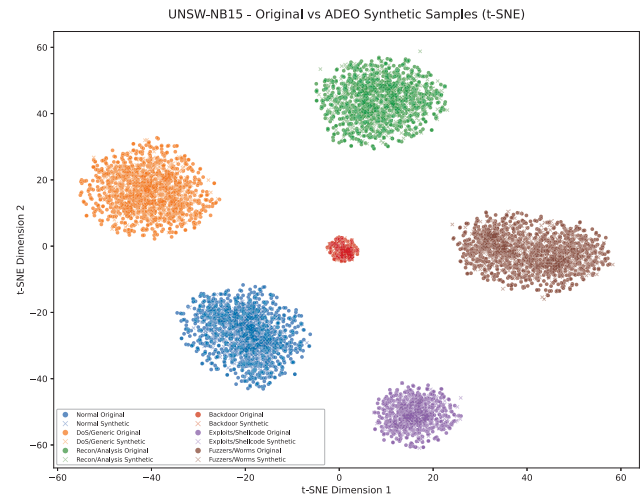


Fig. 10. Schematic of the distribution of synthetic samples generated by ADEO on the UNSW-NB15 dataset compared to the original samples.

cluster structures of the six classes are distinctly defined. Among them, the extremely sparse Backdoor class forms a small, compact cluster in the center, while classes with strong behavioral heterogeneity, such as Fuzzers/Worms and Exploits/Shellcode, exhibit synthetic samples that naturally diffuse along the original distribution without overlapping with the decision regions of adjacent classes. In the Fig. 11, most classes such as Normal and DoS, along with PortScan, form large clusters, while a very small number of classes such as Bot and Web-Attack form small, compact sub-clusters. Even when the expansion ratio reaches hundreds of times, ADEO does not introduce visible noise outliers. In summary, the visualization results across the three datasets consistently demonstrate that the synthetic samples generated by ADEO maintain intra-class compactness while effectively avoiding the noise injection and inter-class blurring issues commonly caused by traditional methods like SMOTE near class boundaries. This provides empirical validation of the advantages of the cluster-first boundary-building, then

TABLE VII
PERFORMANCE COMPARISON OF DIFFERENT OVERSAMPLING METHODS ON THE NSL-KDD DATASET

Comparison Method	Metric	ADEO	Baseline	Diff	W	P	Metric	ADEO	Baseline	Diff	W	P
ADASYN	Accuracy	99.40%±0.0003	98.57%±0.0007	-0.0083	0	0.0625	F1 score	0.9939±0.0003	0.9866±0.0007	-0.0074	0	0.0625
	Precision	99.40%±0.0003	98.87%±0.0005	-0.0053	0	0.0625	FPR	0.0022±0.0001	0.0031±0.0002	0.0009	0	0.0625
	Recall	99.40%±0.0003	98.57%±0.0007	-0.0083	0	0.0625	AUC	0.9992±0.0000	0.9988±0.0005	-0.0004	0	0.0625
Borderline SMOTE	Accuracy	99.40%±0.0003	98.73%±0.0007	-0.0067	0	0.0625	F1 score	0.9939±0.0003	0.9879±0.0007	-0.006	0	0.0625
	Precision	99.40%±0.0003	98.95%±0.0007	-0.0045	0	0.0625	FPR	0.0022±0.0001	0.0028±0.0002	0.0006	0	0.0625
	Recall	99.40%±0.0003	98.73%±0.0007	-0.0067	0	0.0625	AUC	0.9992±0.0000	0.9987±0.0004	-0.0005	0	0.0625
Random Over Sampler	Accuracy	99.40%±0.0003	99.08%±0.0004	-0.0032	0	0.0625	F1 score	0.9939±0.0003	0.9911±0.0004	-0.0028	0	0.0625
	Precision	99.40%±0.0003	99.20%±0.0003	-0.002	0	0.0625	FPR	0.0022±0.0001	0.0021±0.0001	-0.0001	2	0.1875
	Recall	99.40%±0.0003	99.08%±0.0004	-0.0032	0	0.0625	AUC	0.9992±0.0000	0.9995±0.0001	0.0003	0	0.0625
SMOTE	Accuracy	99.40%±0.0003	99.06%±0.0003	-0.0034	0	0.0625	F1 score	0.9939±0.0003	0.9909±0.0003	-0.003	0	0.0625
	Precision	99.40%±0.0003	99.17%±0.0002	-0.0023	0	0.0625	FPR	0.0022±0.0001	0.0022±0.0001	0	7	1
	Recall	99.40%±0.0003	99.06%±0.0003	-0.0034	0	0.0625	AUC	0.9992±0.0000	0.9994±0.0002	0.0002	3	0.3125
SVM SMOTE	Accuracy	99.40%±0.0003	98.84%±0.0005	-0.0056	0	0.0625	F1 score	0.9939±0.0003	0.9890±0.0005	-0.0049	0	0.0625
	Precision	99.40%±0.0003	99.04%±0.0004	-0.0036	0	0.0625	FPR	0.0022±0.0001	0.0026±0.0001	0.0004	0	0.0625
	Recall	99.40%±0.0003	98.84%±0.0005	-0.0056	0	0.0625	AUC	0.9992±0.0000	0.9992±0.0003	0	7	1
GOIO	Accuracy	99.40%±0.0003	98.60%±0.0011	-0.0081	0	0.0625	F1 score	0.9939±0.0003	0.9864±0.0008	-0.0075	0	0.0625
	Precision	99.40%±0.0003	98.72%±0.0007	-0.0068	0	0.0625	FPR	0.0022±0.0001	0.0039±0.0002	0.0017	0	0.0625
	Recall	99.40%±0.0003	98.60%±0.0011	-0.0081	0	0.0625	AUC	0.9992±0.0000	0.9994±0.0000	0.0002	0	0.0625
TMG-GAN	Accuracy	99.40%±0.0003	74.09%±0.0077	-0.2531	0	0.0625	F1 score	0.9939±0.0003	0.6969±0.0083	-0.297	0	0.0625
	Precision	99.40%±0.0003	79.58%±0.0049	-0.1982	0	0.0625	FPR	0.0022±0.0001	0.0888±0.0031	0.0866	0	0.0625
	Recall	99.40%±0.0003	74.09%±0.0077	-0.2531	0	0.0625	AUC	0.9992±0.0000	0.8672±0.0169	-0.132	0	0.0625

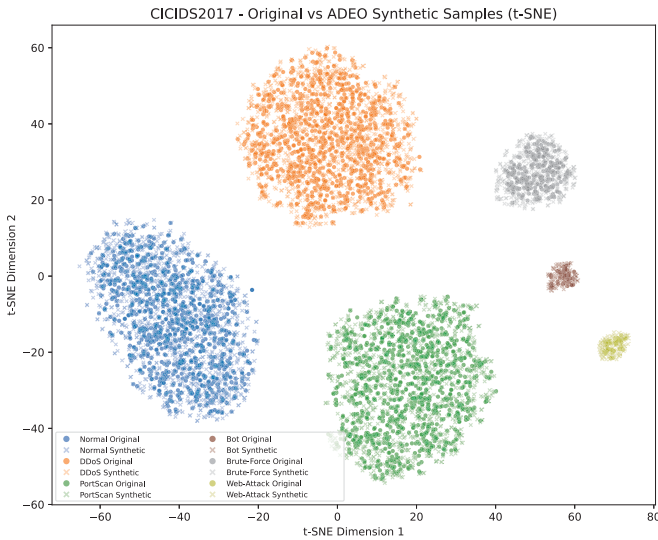


Fig. 11. Schematic of the distribution of synthetic samples generated by ADEO on the CICIDS2017 dataset compared to the original samples.

sampling-within-boundaries mechanism in terms of fidelity and discriminative performance.

4) *Comparative Experimental Analysis*: To validate the effectiveness of the proposed Boundary-Function method (i.e., a method that combines reinforcement learning-based intelligent DBSCAN parameter optimization with uniform oversampling within the boundary function), this paper conducted comparative experiments with various mainstream oversampling methods on three benchmark datasets. The experimental results are shown

in Tables VII, VIII, and IX, respectively, Where P stands for the p-value. Eight methods—including ADEO, RandomOverSampler, SMOTE, BorderlineSMOTE, ADASYN, SVM SMOTE, TMG-GAN, and GOIO—were tested on the NSL-KDD, UNSW-NB15, and CICIDS2017 datasets. All methods employed the same feature engineering workflow and ensemble learning framework (weighted ensemble of LightGBM, XGBoost, and RandomForest). The evaluation process employed five-fold cross-validation, with each fold using 80% of the data for training and 20% for testing. It should be noted that this validation strategy yields only five performance metrics for the Wilcoxon signed-rank test; the small sample size ($n = 5$) results in discrete p-values, with the smallest value being only 0.0625.

To systematically evaluate the performance advantages of the ADEO framework compared to mainstream oversampling methods, this paper adopts five-fold perfect performance (i.e., ADEO outperforms all other methods in all five-fold cross-validation runs, denoted as $W = 0$) as the key indicator for determining a consistent trend of performance superiority among methods, while also incorporating effect size (Diff) and standard deviation into a comprehensive assessment. It should be noted that under the condition of $n = 5$, the theoretical minimum two-sided p-value for the Wilcoxon test is 0.0625, therefore, the p-value is primarily used as a descriptive record of the paired comparison results rather than as a basis for determining statistical significance. On the NSL-KDD dataset, ADEO consistently achieved 99.40% across all four metrics—accuracy, precision, recall, and F1 score (F1 score = 0.9939)—demonstrating a consistent performance advantage over ADASYN, BorderlineSMOTE, RandomOverSampler, SMOTE, SVM SMOTE, and GOIO ($W = 0$, $p = 0.0625$), with performance improvements

TABLE VIII
PERFORMANCE COMPARISON OF DIFFERENT OVERSAMPLING METHODS ON THE UNSW-NB15 DATASET

Comparison Method	Metric	ADEO	Baseline	Diff	W	P	Metric	ADEO	Baseline	Diff	W	P
ADASYN	Accuracy	99.05%±0.0011	97.29%±0.0002	-0.0176	0	0.0625	F1 score	0.9903±0.0012	0.9748±0.0002	-0.0154	0	0.0625
	Precision	99.02%±0.0012	97.91%±0.0002	-0.0111	0	0.0625	FPR	0.0059±0.0008	0.0047±0.0000	-0.0012	0	0.0625
	Recall	99.05%±0.0011	97.29%±0.0002	-0.0176	0	0.0625	AUC	0.9988±0.0002	0.9959±0.0001	-0.0028	0	0.0625
Borderline SMOTE	Accuracy	99.05%±0.0011	97.15%±0.0002	-0.019	0	0.0625	F1 score	0.9903±0.0012	0.9724±0.0002	-0.0179	0	0.0625
	Precision	99.02%±0.0012	97.64%±0.0002	-0.0138	0	0.0625	FPR	0.0059±0.0008	0.0050±0.0000	-0.0009	1	0.125
	Recall	99.05%±0.0011	97.15%±0.0002	-0.019	0	0.0625	AUC	0.9988±0.0002	0.9953±0.0002	-0.0035	0	0.0625
Random Over Sampler	Accuracy	99.05%±0.0011	96.84%±0.0002	-0.022	0	0.0625	F1 score	0.9903±0.0012	0.9761±0.0001	-0.0142	0	0.0625
	Precision	99.02%±0.0012	98.69%±0.0001	-0.0033	0	0.0625	FPR	0.0059±0.0008	0.0053±0.0000	-0.0006	3	0.3125
	Recall	99.05%±0.0011	96.84%±0.0002	-0.022	0	0.0625	AUC	0.9988±0.0002	0.9962±0.0001	-0.0026	0	0.0625
SMOTE	Accuracy	99.05%±0.0011	97.10%±0.0002	-0.0195	0	0.0625	F1 score	0.9903±0.0012	0.9766±0.0001	-0.0137	0	0.0625
	Precision	99.02%±0.0012	98.45%±0.0001	-0.0057	0	0.0625	FPR	0.0059±0.0008	0.0049±0.0000	-0.001	0	0.0625
	Recall	99.05%±0.0011	97.10%±0.0002	-0.0195	0	0.0625	AUC	0.9988±0.0002	0.9962±0.0001	-0.0026	0	0.0625
SVMS MOTE	Accuracy	99.05%±0.0011	97.00%±0.0008	-0.0204	0	0.0625	F1 score	0.9903±0.0012	0.9753±0.0003	-0.015	0	0.0625
	Precision	99.02%±0.0012	98.24%±0.0009	-0.0078	0	0.0625	FPR	0.0059±0.0008	0.0059±0.0003	0	7	1
	Recall	99.05%±0.0011	97.00%±0.0008	-0.0204	0	0.0625	AUC	0.9988±0.0002	0.9943±0.0003	-0.0044	0	0.0625
GOIO	Accuracy	99.05%±0.0011	99.13%±0.0008	0.0008	1	0.125	F1 score	0.9903±0.0012	0.9918±0.0008	0.0015	0	0.0625
	Precision	99.02%±0.0012	99.31%±0.0006	0.0029	0	0.0625	FPR	0.0059±0.0008	0.0039±0.0009	-0.002	1	0.125
	Recall	99.05%±0.0011	99.13%±0.0008	0.0008	1	0.125	AUC	0.9988±0.0002	0.9998±0.0000	0.001	0	0.0625
TMG-GAN	Accuracy	99.05%±0.0011	97.29%±0.0003	-0.0176	0	0.0625	F1 score	0.9903±0.0012	0.9759±0.0006	-0.0144	0	0.0625
	Precision	99.02%±0.0012	98.19%±0.0023	-0.0083	0	0.0625	FPR	0.0059±0.0008	0.0054±0.0009	-0.0005	5	0.625
	Recall	99.05%±0.0011	97.29%±0.0003	-0.0176	0	0.0625	AUC	0.9988±0.0002	0.9712±0.0060	-0.0276	0	0.0625

TABLE IX
PERFORMANCE COMPARISON OF DIFFERENT OVERSAMPLING METHODS ON THE CICIDS2017 DATASET

Comparison Method	Metric	ADEO	Baseline	Diff	W	P	Metric	ADEO	Baseline	Diff	W	P
ADASYN	Accuracy	99.81%±0.0004	99.28%±0.0006	-0.0053	0	0.0625	F1 score	0.9980±0.0004	0.9944±0.0004	-0.0036	0	0.0625
	Precision	99.80%±0.0005	99.66%±0.0003	-0.0014	0	0.0625	FPR	0.0009±0.0002	0.0009±0.0001	0.0001	5	0.625
	Recall	99.81%±0.0004	99.28%±0.0006	-0.0053	0	0.0625	AUC	0.9981±0.0021	0.9985±0.0011	0.0005	7	1
Borderline SMOTE	Accuracy	99.81%±0.0004	99.24%±0.0021	-0.0057	0	0.0625	F1 score	0.9980±0.0004	0.9939±0.0015	-0.0041	0	0.0625
	Precision	99.80%±0.0005	99.60%±0.0008	-0.0021	0	0.0625	FPR	0.0009±0.0002	0.0011±0.0003	0.0002	5	0.625
	Recall	99.81%±0.0004	99.24%±0.0021	-0.0057	0	0.0625	AUC	0.9981±0.0021	0.9968±0.0028	-0.0012	4	0.4375
Random Over Sampler	Accuracy	99.81%±0.0004	99.75%±0.0004	-0.0006	1	0.125	F1 score	0.9980±0.0004	0.9978±0.0003	-0.0002	4	0.4375
	Precision	99.80%±0.0005	99.84%±0.0002	0.0003	2	0.1875	FPR	0.0009±0.0002	0.0003±0.0001	-0.0005	0	0.0625
	Recall	99.81%±0.0004	99.75%±0.0004	-0.0006	1	0.125	AUC	0.9981±0.0021	0.9987±0.0020	0.0006	0	0.0625
SMOTE	Accuracy	99.81%±0.0004	99.48%±0.0003	-0.0033	0	0.0625	F1 score	0.9980±0.0004	0.9961±0.0003	-0.0019	0	0.0625
	Precision	99.80%±0.0005	99.80%±0.0002	-0.0001	5	0.625	FPR	0.0009±0.0002	0.0007±0.0001	-0.0002	1	0.125
	Recall	99.81%±0.0004	99.48%±0.0003	-0.0033	0	0.0625	AUC	0.9981±0.0021	0.9960±0.0058	-0.0021	6	0.8125
SVM SMOTE	Accuracy	99.81%±0.0004	99.79%±0.0003	-0.0002	4	0.4375	F1 score	0.9980±0.0004	0.9980±0.0002	0	6	0.8125
	Precision	99.80%±0.0005	99.83%±0.0002	0.0002	5	0.625	FPR	0.0009±0.0002	0.0003±0.0000	-0.0005	0	0.0625
	Recall	99.81%±0.0004	99.79%±0.0003	-0.0002	4	0.4375	AUC	0.9981±0.0021	0.9968±0.0054	-0.0012	6	0.8125
GOIO	Accuracy	99.81%±0.0004	99.35%±0.0006	-0.0046	0	0.0625	F1 score	0.9980±0.0004	0.9927±0.0008	-0.0054	0	0.0625
	Precision	99.80%±0.0005	99.21%±0.0011	-0.006	0	0.0625	FPR	0.0009±0.0002	0.0012±0.0001	0.0004	0	0.0625
	Recall	99.81%±0.0004	99.35%±0.0006	-0.0046	0	0.0625	AUC	0.9981±0.0021	0.9998±0.0000	0.0017	0	0.0625
TMG-GAN	Accuracy	99.81%±0.0004	95.30%±0.0039	-0.0451	0	0.0625	F1 score	0.9980±0.0004	0.9530±0.0042	-0.045	0	0.0625
	Precision	99.80%±0.0005	95.60%±0.0048	-0.0421	0	0.0625	FPR	0.0009±0.0002	0.0152±0.0009	0.0143	0	0.0625
	Recall	99.81%±0.0004	95.30%±0.0039	-0.0451	0	0.0625	AUC	0.9981±0.0021	0.9283±0.0165	-0.0697	0	0.0625

ranging from 0.32% to 0.83%. Its advantage over TMG-GAN is particularly pronounced, with accuracy and recall leading by 25.31%. On the UNSW-NB15 dataset, which features a more extreme class distribution, ADEO (accuracy = 99.05%) demonstrated a consistent performance advantage of 1.37%–2.20% over the aforementioned six methods on key metrics ($W = 0$, $p = 0.0625$), while GOIO exhibits a consistent, albeit slight,

advantage in precision and AUC ($W = 0$, $p = 0.0625$), forming the top tier alongside ADEO. On the CICIDS2017 dataset, ADEO achieved an overall performance of accuracy = 99.81%, F1 score = 0.9980, and FPR = 0.0009, demonstrating a consistent performance advantage over ADASYN, BorderlineSMOTE, SMOTE, GOIO, and TMG-GAN ($W = 0$, $p = 0.0625$), while the gap with RandomOverSampler and SVMSMOTE

TABLE X
CALCULATED COST OF THE PIPELINE(SECOND)

Dataset	Feature Engineering	DQN Parameter Optimization	DBSCAN Clustering	Geometric Calculation	Boundary Oversampling	Ensemble Training
NSL-KDD	14.2924±0.1844	12.6999±4.0141	50.1590±22.8200	0.8669±0.0132	0.1205±0.0071	9.5599±0.1948
UNSW-NB15	514.4141±16.9325	5.3371±0.9658	937.0191±463.5574	15.6365±0.1037	2.6831±0.0493	460.5972±10.7589
CICIDS2017	649.7951±84.4748	3.6393±0.7101	277.3480±74.5719	18.0891±0.9936	4.2398±0.0658	781.0330±62.1288

narrows due to the saturation of baseline performance under sufficient sample sizes ($W \geq 1$, p-values ranging from 0.125 to 0.8125, with no consistent performance advantage observed). In summary, ADEO demonstrates outstanding performance in extremely imbalanced scenarios and remains competitive with state-of-the-art methods under large-sample conditions, fully validating the effectiveness and generalization capability of the proposed framework.

5) *Analysis of Pipeline Computational Cost and Latency*: To comprehensively evaluate the practical feasibility of deploying the proposed framework, this section analyzes the system's computational costs from the perspectives of training overhead and inference latency. The training overhead for each stage is shown in Table X.

The time required for feature engineering is directly proportional to the size of the dataset, increasing from 14.29 s for NSL-KDD to 649.80 s for CICIDS2017. Due to this stage includes feature selection based on mutual information, whose computational complexity grows linearly with the number of samples.

The time required for DQN parameter optimization is independent of the dataset size. In fact, it shows an inverse trend (NSL-KDD 12.70 s > UNSW-NB15 5.34s > CICIDS2017 3.64s). This results from the DQN stage employs a fixed sub-sampling strategy, and the computational load does not depend on the size of the full dataset. NSL-KDD takes the longest time and has the largest standard deviation (± 4.01). This results from its feature space is more compact, under the same ε parameter, the DBSCAN neighborhood search involves a higher density of points, resulting in longer processing times per clustering iteration.

DBSCAN clustering is the most computationally intensive stage in the training pipeline and exhibits significant variability (with a standard deviation as high as 463.56 s for UNSW-NB15). UNSW-NB15 took the longest time (937.02 s) rather than CICIDS2017, which has a larger sample size (277.35 s). This is directly related to the feature space dimension, data distribution density, and the optimal ε parameter selected by DQN—smaller ε values reduce the neighborhood search range, thereby accelerating clustering.

The computational overhead of geometric computations and boundary oversampling is minimal, accounting for only 0.5% – 1.3% of the total training time across the three datasets, indicating that the geometric boundary mechanism proposed in this paper does not introduce a significant additional computational burden.

The time required to train the ensemble model is directly proportional to the volume of the oversampled training data,

TABLE XI
LATENCY ANALYSIS

Batch Size	Metrics	NSL-KDD	UNSW-NB15	CICIDS2017
1	Per-sample Latency(ms)	128.4661	110.1633	118.7412
	Throughput(samples/s)	7	9	8
10	Per-sample Latency(ms)	11.3551	10.0674	11.7121
	Throughput(samples/s)	88	99	85
100	Per-sample Latency(ms)	1.0200	1.0120	1.1664
	Throughput(samples/s)	980	988	857
1000	Per-sample Latency(ms)	0.1076	0.1187	0.1282
	Throughput(samples/s)	9293	8423	7798
10000	Per-sample Latency(ms)	0.0200	0.0229	0.0259
	Throughput(samples/s)	50085	43749	38562

increasing from 9.56 s for NSL-KDD to 781.03 s for CICIDS2017, because all five base learners (LightGBM, XGBoost, CatBoost, Random Forest, and Extra Trees) must be trained on the full dataset.

To verify the feasibility of deploying the proposed method in real-world network environments, systematic tests of its inference performance is conducted. In actual online deployment, the inference pipeline consists of only two lightweight stages: (1) applying feature transformations to input samples using the normalizer and feature selector fitted during the training phase, and (2) making predictions using the trained integrated classifier. The modules involved in the training phase—DQN parameter optimization, DBSCAN clustering, geometric boundary fitting, density denoising, and boundary oversampling—are not utilized during inference.

The latency analysis is shown in Table XI, the inference latency performance of the three datasets is highly consistent across different batch sizes. When batch = 1, the latency per sample is approximately 110–128 ms, primarily due to the fixed initialization overhead of feature transformations. As the batch size increases, this fixed overhead is effectively spread out, and the latency per sample drops to 0.108–0.128 ms when batch = 1,000. In the most common batch processing scenario for real-world deployment (batch = 10,000), the per-sample latency for all three datasets is below 0.03 ms, with throughput reaching 38,563–50,086 samples/s. Mainstream network monitoring systems offer varying levels of processing capability: Zeek is designed for 10 Gbps network traffic monitoring in typical deployments (<https://wlcg-soc-wg-doc.web.cern.ch/>) and can scale to 100GE links in multi-core cluster configurations (<https://docs.zeek.org/en/master/about.html>). Suricata achieves approximately 30–35Gbps of lossless throughput on standard

network cards (with Napatech SmartNICs boosting Suricata's performance to 100 Gbps), and this can be further enhanced with hardware acceleration. In terms of flow record generation rates, a 1 Gbps link typically generates thousands to tens of thousands of flow records per second, while a 10 Gbps link can generate tens of thousands to over a hundred thousand. This framework achieves an inference throughput of 38,563–50,086 samples/s when batch = 10,000, which meets the real-time processing requirements of the typical scenarios described above.

V. CONCLUSION

This paper proposes an adaptive DBSCAN-enhanced oversampling framework (ADEO) to address the severe challenge of extreme class imbalance in network intrusion detection systems. ADEO pioneers the integration of deep reinforcement learning with density-based clustering, automatically optimizing DBSCAN parameters via DQN while generating high-quality synthetic samples constrained by geometric boundaries. To further enhance the framework's adaptability to diverse attack types, we designed a hierarchical class mapping strategy and dataset-specific feature engineering modules. This enables the framework to simultaneously optimize detection performance for extremely rare classes while maintaining overall classification accuracy. Additionally, we comprehensively evaluated the proposed method on three representative network intrusion detection datasets. Experiments demonstrate that ADEO comprehensively outperforms mainstream oversampling methods across all performance metrics, achieving particularly significant improvements in detecting ultra-rare classes, proving the superiority of the proposed approach.

In future research, we will conduct an in-depth study of the effectiveness of adaptive boundary detection mechanisms in classifying encrypted traffic for network intrusion detection. By incorporating representation learning, to automatically extract high-quality, low-dimensional features from raw network traffic, thereby enhancing the system's ability to detect unknown attacks and complex patterns.

ACKNOWLEDGMENT

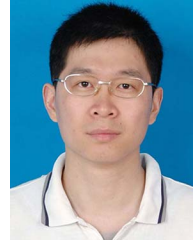
All authors thank the editors and anonymous reviewers for their valuable comments and precious suggestions, this work was also supported by the Zhejiang Key Laboratory of Intelligent Education Technology and Application.

REFERENCES

- [1] L. Yang et al., "Enhanced model poisoning attack and multi-strategy defense in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 3877–3892, 2025.
- [2] Y. Yang et al., "UWBAD: Towards effective and imperceptible jamming attacks against UWB ranging systems with COTS chips," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA: Association for Computing Machinery, 2024, pp. 3376–3390.
- [3] Y. Zhang et al., "A practical DoS attack on commercial UWB ranging systems," *IEEE Trans. Mobile Comput.*, vol. 24, no. 10, pp. 10492–10509, Oct. 2025.
- [4] Y. Yang et al., "Automating function-level TARA for automotive full-lifecycle security," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, USA, 2026, pp. 1–18.
- [5] C. Qiu et al., "Mining multi-scale spatial-frequency clues for unsupervised intrusion detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 11447–11461, 2025.
- [6] J. Dong, H. Chen, S. Shen, H. Xu, and Z. Liu, "Interpretable intrusion detection for IoT security: A shap-enhanced ngboost model," *Comput. Netw.*, vol. 275, 2026, Art. no. 111937.
- [7] M. S. Milosevic and V. M. Ciric, "Extreme minority class detection in imbalanced data for network intrusion," *Comput. Secur.*, vol. 123, 2022, Art. no. 102940.
- [8] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1156–1167, 2024.
- [9] M. A. Talukder et al., "Machine learning-based network intrusion detection for big and imbalanced data using oversampling, stacking feature embedding and feature extraction," *J. Big Data*, vol. 11, no. 1, 2024, Art. no. 33.
- [10] L. Jin et al., "Intrusion detection for future its: Integrated knowledge graph and artificial intelligence," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 10, pp. 14680–14689, Oct. 2025.
- [11] X. Ma et al., "On the influence of data resampling for deep learning-based log anomaly detection: Insights and recommendations," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 243–261, Jan. 2025.
- [12] T.-T.-H. Le, Y. Shin, M. Kim, and H. Kim, "Towards unbalanced multiclass intrusion detection with hybrid sampling methods and ensemble classification," *Appl. Soft Comput.*, vol. 157, 2024, Art. no. 111517.
- [13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [14] Z. Qin, Y. Liu, T. Wang, L. Chen, and Y. Song, "Effective sample generation for industrial control systems: A GAN-based approach," *IEEE Trans. Ind. Informat.*, vol. 22, no. 2, pp. 706–716, Feb. 2026.
- [15] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IEEE World Congr. Comput. Intell.)*, Hong Kong, SAR, China, 2008, pp. 1322–1328.
- [16] G. Douzas and F. Bacao, "Geometric SMOTE a geometrically enhanced drop-in replacement for SMOTE," *Inf. Sci.*, vol. 501, pp. 118–135, 2019.
- [17] J. Manokaran, G. Vairavel, and J. Vijaya, "PPFCM-SMOTE: A novel balancing system for anomaly detection in IoT edge using probabilistic possibilistic fuzzy clustering and SMOTE," *Int. J. Inf. Technol.*, pp. 1–20, 2024.
- [18] Z. Xu, D. Shen, Y. Kou, and T. Nie, "A synthetic minority oversampling technique based on Gaussian mixture model filtering for imbalanced data classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3740–3753, Mar. 2024.
- [19] P. Mondal, F. Ansari, and S. Das, "CCO: A cluster core-based oversampling technique for improved class-imbalanced learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 9, no. 2, pp. 1153–1165, Apr. 2025.
- [20] P. Sun, Z. Wang, L. Jia, and L. Wang, "An oversampling technique based on noise detection and geometry," *Appl. Soft Comput.*, vol. 170, 2025, Art. no. 112718.
- [21] H. Sun, J. Li, and X. Zhu, "A novel expandable borderline SMOTE oversampling method for class imbalance problem," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 5, pp. 2183–2199, May 2025.
- [22] H. Wang, F. Kandah, T. Mendis, and L. Medury, "Clustering-based intrusion detection system meets multicritics generative adversarial networks," *IEEE Internet Things J.*, vol. 12, no. 11, pp. 16112–16128, Jun. 2025.
- [23] Y. Liu and Q. Liu, "SMOTE oversampling algorithm based on generative adversarial network," *Cluster Comput.*, vol. 28, no. 4, 2025, Art. no. 271.
- [24] S. Hong, S. An, and J.-J. Jeon, "Improving SMOTE via fusing conditional VAE for data-adaptive noise filtering," *Appl. Intell.*, vol. 55, no. 12, 2025, Art. no. 841.
- [25] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, Ottawa, ON, Canada, 2009, pp. 1–6.
- [26] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf.*, Canberra, Australia, 2015, pp. 1–6.
- [27] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, Funchal, Portugal, 2018, pp. 108–116.
- [28] S. Ren, J. Ding, C. Yang, and Y.-m. Cheung, "GOIO: Generative oversampling approach to class imbalance and overlap of tabular data," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 11, pp. 6450–6463, Nov. 2025.



Jingnan Dong (Member, IEEE) received the BS degree in computer science and technology from Zhengzhou University, Henan, China, in 2016, and the PhD degree in computer science and technology from Chongqing University of Posts and Telecommunications, Chongqing, China, in 2023. He is currently a associate professor with the School of Information Engineering, Huzhou Normal University, Zhejiang, China. His research interests include network intrusion detection, encrypted traffic analysis.



Jun Liu (Member, IEEE) received the PhD degree in computer science and technology from Chongqing University, Chongqing, China, in 2008 and 2016, respectively. He is currently an associate professor with the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing. He has authored or coauthored more than 20 research papers in a series of journals and conference proceedings such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Fuzzy Systems*, *IEEE Transactions on Computational Social Systems*, *IEEE Internet of Things Journal*, and *ESWA*. His research interests include data security, Big Data analytics, and deep learning.



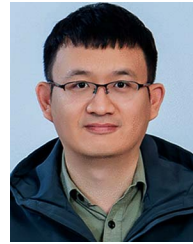
Boran Zhang (Student Member, IEEE) is currently working toward the BS degree in computer science and technology with Huzhou Normal University, Zhejiang, China. His research interests include network intrusion detection, encrypted traffic analysis, and neural networks. He was awarded the National Second Prize at the 22nd China National College Students Information Security and Countermeasure Technology Competition in 2025. He is student member of the Chinese Computer Federation.



Guangxia Xu (Senior Member, IEEE) received the MS and PhD degrees in computer science from Chongqing University, Chongqing, China, in 2006 and 2011, respectively. She is currently a professor and the PhD supervisor with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou, China. She is also with the Department of New Networks, Peng Cheng Laboratory. Her research interests include information security and network management. She is a committee member at the Blockchain of CCF, and ACM member.



Shigen Shen (Senior Member, IEEE) received the PhD degree in pattern recognition and intelligent systems from Donghua University, Shanghai, China, in 2013. He is a professor with the School of Information Engineering, Huzhou Normal University, Huzhou, China. He has authored or coauthored more than 100 technical articles, including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Computational Social Systems*, and *IoT-J*. His research interests include the Internet of Things, cybersecurity, edge computing, and game theory.



Zhiquan Liu (Senior Member, IEEE) received the BS degree from the School of Science, Xidian University, Xi'an, China, in 2012, and the PhD degree from the School of Computer Science and Technology, Xidian University in 2017. He is currently a full professor with the College of Cyber Security, Jinan University, Guangzhou, China. His research interests include security, trust, privacy, and intelligence in vehicular networks. He is an the area editor or associate editor of multiple SCI-index journals, such as *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Industrial Informatics*, *IEEE Internet of Things Journal*, *Information Fusion*, *Computer Networks*, *Journal of Systems Architecture*, and *Ad Hoc Networks*. His homepage is <https://www.zqliu.com>.